

استاد: دکتر داوودآبادی

نام درس: مبانی پردازش زبان و گفتار

گزارش: HW3

نام و نام‌خانوادگی: نیوشا یقینی

شماره دانشجویی: ۹۸۵۲۲۳۴۶

اردیبهشت ۱۴۰۴

Content Table

1. **Question 1** 3

1) Step 1: Install and Import Python Libraries 5

2) Step 2: Download and Read Data 5

3) Step 3: Train Test Split 6

4) Step 4: Convert Pandas Dataframe to Hugging Face Dataset 6

5) Step 5: Tokenize Text 7

6) Step 6: Load Pretrained Model 12

(٧ Step 7: Set Training Argument 13

8) Step 8: Set Evaluation Metrics 15

9) Step 9: Train Model Using Transformer Trainer 16

- Base Model (Repeated) 26
- Improved Model 26
- Optuna (Top 10) 27

10) Step 10: Make Predictions for Text Classification 27

11) Step 11: Model Performance Evaluation 30

12) Step 12: Save and Load Model 30

2. **Question 2** 31

1) Step 1: Installing Libraries 31

2) Step 2: Loading The Model 31

3) Step 3: Checking examples 32

4) Step 4 - Method 1: Cosine similarity 33

5) Step 5 – Method 2: Word analogy task 35

6) Step 6 – Method 3: Debiasing word vectors (OPTIONAL/UNGRADED) 37

۱. سوال ۱

تا به اکنون شما با ترنسفورمرها و نیز تسک طبقه‌بندی متون مثل sentiment analysis آشنا شده‌اید. در این تمرین قصد داریم تا شما را با fine-tune کردن یک مدل BERT از پیش آموزش دیده شده برای تسک sentiment analysis با استفاده از transformer PyTorch Trainer از Hugging Face آشنا کنیم. لطفاً به نوت بوک این سوال یعنی فایل (Assignment 1.ipynb) مراجعه نمایید و قسمت‌های خواسته‌شده را تکمیل فرمایید. تمامی توضیحات مربوط به هر بخش در اختیار شما قرار گرفته است. (توجه! بهتر است در گوگل کولب اجرا کنید.)

در گام اول بایستی دیتاست را از این بخش دانلود کنید (از دیتاست amazon_cells_labelled.txt استفاده کنید):

https://archive.ics.uci.edu/dataset/331/sentiment+labelled+sentences

برای بخش آموزش و ست کردن پارامترهای مدل، پارامترهای مختلفی را تست و ارزیابی کنید تا به یک مینیمم محلی بهینه دست یابید. (البته دقت مدل با پارامترهای پیش‌فرض بالا، ۹۰ درصد خواهد بود)

به عنوان کار تشویقی و امتیازی در این قسمت می‌توانید استراتژی‌های مختلفی را برای hyperparameter tuning استفاده کنید. (مثل grid search، optuna، روش‌های بیزین و ...)

بعد از آموزش مدل، نوبت می‌رسد به ارزیابی عملکرد مدل. بدین منظور لطفاً متریک‌های مختلفی مثل دقت، فراخوانی، f1 score، AUC و ... را محاسبه و در گزارش خود ذکر کنید. راستی، از کجا می‌فهمید که مدل شما underfit یا overfit نشده است؟

کار امتیازی: در این تمرین ما از tokenizer و مدل sequence classifier برت (BERT) برای قسمت‌های مختلف تسک استفاده کردیم، شما می‌توانید از بین لیست tokenizerها و مدل‌های موجود در hugging face، موارد مختلفی را تست کنید و نتایج آن‌ها را با هم مقایسه کنید.

توضیحات اولیه:

Hugging Face سه روش برای fine-tune کردن یک مدل طبقه‌بندی متن پیش‌آموخته فراهم می‌کند:

TensorFlow Keras, PyTorch, transformer trainer.

۱. استفاده از TensorFlow /Keras
برای کسانی که با TensorFlow کار می‌کنند، می‌توانند مدل را با استفاده از TFTrainer یا APIهای استاندارد Keras آموزش دهند.
۲. استفاده مستقیم از PyTorch
یعنی خودتان حلقه‌های آموزش (training loop)، به‌روزرسانی وزن‌ها، محاسبه loss و ... را به صورت دستی در PyTorch پیاده‌سازی می‌کنید.
۳. استفاده از transformers.Trainer یا Trainer در PyTorch
این روش ساده‌ترین و کامل‌ترین است چون Hugging Face خودش همه‌چیز (training loop، ارزیابی، ذخیره مدل و...) را هندل می‌کند. کافیست مدل، داده، و تنظیمات را بدهید.

در این سوال ما با نوعی از ترنسفورمرها قرار است کار کنیم، ترنسفورمر (Transformer) یک معماری مدل یادگیری عمیق است که بر پایه‌ی مکانیزم Self-Attention ساخته شده و هدفش این است که مدل بتواند معنای هر کلمه رو در رابطه با تمام کلمات جمله یا متن درک کند، نه فقط بر اساس ترتیب. Self-Attention به مدل این امکان رو می‌دهد که در هنگام پردازش هر کلمه، به تمام کلمات دیگر جمله هم نگاه کند و تشخیص بدهد که هر کلمه، چقدر به بقیه کلمات مرتبط است.

در این سوال قرار است ما با مدل Bert کار کنیم. این مدل در زمان آموزش اولیه خود (pretraining)، دو وظیفه‌ی اصلی را یاد می‌گیرد که هر دو برای درک عمیق‌تر زبان طراحی شده‌اند:

- Masked Language Modeling (MLM):

بطور کلی در این وظیفه، برخی از توکن‌های ورودی به‌صورت تصادفی با یک توکن خاص به نام [MASK] جایگزین می‌شوند. مدل باید **پیش‌بینی کند که توکن واقعی در جای آن چیست**، مدل با این کار یاد می‌گیرد که **معنای هر کلمه را در بافت (context) آن جمله** درک کند، نه صرفاً به ترتیب جلو برود. برخلاف مدل‌های قدیمی‌تر که فقط از چپ به راست یا راست به چپ یاد می‌گرفتند، BERT به‌صورت دوطرفه (bidirectional) آموزش می‌بیند، یعنی هم از قبل و هم از بعد توکن‌های ماسک‌شده استفاده می‌کند تا معنی دقیق‌تری را یاد بگیرد.

مثال:

ورودی به مدل = I love [MASK] books:

مدل باید یاد بگیرد که در جای [MASK] احتمالاً کلمه "reading" یا "science" قرار بگیرد.

- Next Sentence Prediction (NSP)

بطور کلی مدل دو جمله دریافت می‌کند و باید تشخیص دهد که آیا **جمله دوم واقعاً ادامه جمله اول هست یا نه**. این وظیفه کمک می‌کند مدل **روابط معنایی و ترتیبی بین جملات** را یاد بگیرد، که برای وظایفی مثل پاسخ به سؤال یا خلاصه‌سازی متن بسیار مفید است.

مثال:

جمله اول = He went to the store

جمله دوم (واقعی) = He bought some milk

Label = IsNext

جمله اول = He went to the store

جمله دوم (تصادفی) = The Eiffel Tower is in Paris

Label = NotNext

مراحل پیاده‌سازی:

1) Step 1: Install and Import Python Libraries

ابتدا کتابخانه‌های مورد نیاز پروژه را نصب کردیم.

در این کد، کتابخانه‌های لازم برای **آموزش مدل** BERT و **تحلیل احساسات** نصب می‌شوند. از transformers و datasets برای بارگذاری مدل و داده استفاده می‌شود، evaluate برای ارزیابی عملکرد مدل به کار گرفته می‌شود، و pandas و numpy برای پردازش اولیه داده‌ها مورد استفاده قرار می‌گیرند. از AutoTokenizer و AutoModelForSequenceClassification برای آماده‌سازی مدل و متن استفاده می‌شود، و Trainer برای آموزش مدل بدون نیاز به نوشتن حلقه آموزشی دستی به کار می‌رود. همچنین Dataset برای تبدیل داده به فرمت استاندارد Hugging Face استفاده می‌شود و TextClassificationPipeline برای پیش‌بینی نهایی مورد استفاده قرار می‌گیرد.

```
# Install libraries
!pip install transformers datasets evaluate

# Data processing
import pandas as pd
import numpy as np

# Modeling
import tensorflow as tf
from transformers import AutoTokenizer, AutoModelForSequenceClassification, \
    TrainingArguments, Trainer, EarlyStoppingCallback, TextClassificationPipeline

# Hugging Face Dataset
from datasets import Dataset

# Model performance evaluation
import evaluate
```

2) Step 2: Download and Read Data

اجرای پروژه روی لپ‌تاپ سریع‌تر، پایدارتر و قابل کنترل‌تر بود، و منابع سیستم نیز برای آموزش مدل BERT کافی بودند همچنین چرا که نسخه رایگان کولب محدودیت‌هایی از جمله قطعی GPU و منابع کمتر دارد، بنابراین کد بخش مرتبط با استفاده از Google Colab را کامنت کردیم.

از سایت گفته‌شده دیتاست amazon_cells_labelled را دانلود کردیم. این فایل از دیتاست معروف UCI به نام **Sentiment Labelled Sentences Dataset** گرفته شده و در واقع یکی از سه فایل متنی موجود در این مجموعه است.

دیتاست مورد نظر دارای ۲ ستون review و label هست، که ستون اول جملات واقعی از نقدهای کاربران درباره محصولات آمازون (به‌ویژه موبایل و لوازم جانبی) است، و ستون دوم یک عدد (منفی = ۰، مثبت = ۱) است.

این دیتاست شامل ۱۰۰۰ نمونه جمله هست که بطور متوازن ۵۰۰ نظر مثبت و ۵۰۰ نظر منفی دارد، ستون اول از نوع object و ستون دوم از نوع int64 هست، و تمام سلول‌ها non-null هستند.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   review  1000 non-null   object
 1   label   1000 non-null   int64
dtypes: int64(1), object(1)
memory usage: 15.8+ KB
```

```
label
0    500
1    500
Name: count, dtype: int64
```

3) Step 3: Train Test Split

در **مرحله ۳**، داده‌ها را به دو بخش تقسیم می‌کنیم؛ به‌طوری‌که **۸۰ درصد به‌عنوان داده آموزشی** (Train Data) و **۲۰ درصد به‌عنوان داده آزمایشی** (Train Data) در نظر گرفته می‌شود.

با استفاده از متد sample و تعیین مقدار frac=0.8، به‌صورت تصادفی ۸۰ درصد داده‌ها نمونه‌برداری می‌شود. تنظیم random_state=42 باعث می‌شود که نتیجه نمونه‌گیری هر بار به‌صورت یکنواخت و قابل بازتولید باشد.

سپس با **حذف داده‌های آموزشی** از مجموعه اصلی، ۲۰ درصد باقی‌مانده به‌عنوان **داده آزمایشی** در نظر گرفته می‌شود.

```
# Training dataset
train_data = amz_review.sample(frac=0.8, random_state=42)

# Testing dataset
test_data = amz_review.drop(train_data.index)

# Check the number of records in training and testing dataset.
print(f'The training dataset has {len(train_data)} records.')
print(f'The testing dataset has {len(test_data)} records.')
```

که در نتیجه ۸۰٪ از داده‌ها که مقدار آنها ۸۰۰تا هست بصورت تصادفی به عنوان آموزش برداشته می‌شوند.

و با حذف آن‌ها از دیتافریم اصلی، ۲۰٪ باقیمانده که مقدار آن ۲۰۰تا هست برای تست در نظر گرفته می‌شوند.

```
The training dataset has 800 records.
The testing dataset has 200 records.
```

4) Step 4: Convert Pandas Dataframe to Hugging Face Dataset

در مرحله ۴، داده‌های آموزشی و آزمایشی با استفاده از تابع Dataset.from_pandas از قالب DataFrame پانداس به فرمت Dataset کتابخانه Hugging Face تبدیل می‌شوند. این ساختار به‌صورت memory-mapped روی دیسک ذخیره می‌شود و محدودیتی از نظر RAM ندارد، که برای پردازش داده‌های بزرگ بسیار مفید است. همچنین، طول Dataset جدید با تعداد رکوردهای دیتافریم اولیه (۸۰۰ تا) برابر خواهد بود.

```
# Convert pyhton dataframe to Hugging Face arrow dataset

hg_train_data = Dataset.from_pandas(train_data)
hg_test_data = Dataset.from_pandas(test_data)
```

عبارت hg_train_data[0] اولین رکورد Dataset را به‌صورت یک دیکشنری بازمی‌گرداند که شامل سه کلید است: review (متن نقد)، label (برچسب احساسی جمله)، و __index_level_0__ (اندیس اصلی رکورد که به‌صورت خودکار از دیتافریم پانداس اضافه شده است.)

```
# Length of the Dataset
print(f'The length of hg_train_data is {len(hg_train_data)}.\n')

# Check one review
hg_train_data[0]
```

```
The length of hg_train_data is 800.


{'review': 'Thanks again to Amazon for having the things I need for a good price!',
 'label': 1,
 '__index_level_0__': 521}
```

5) Step 5: Tokenize Text

در مرحله ۵، متن نقدها با استفاده از **توکنایزر** به اعداد (توکن‌ها) تبدیل می‌شود تا به‌عنوان ورودی مدل NLP قابل استفاده باشد.

دستور AutoTokenizer.from_pretrained("bert-base-cased") برای دانلود واژگان مدل پیش‌آموخته bert-base-cased استفاده می‌شود، به این معنا که متن دقیقاً به همان شیوه‌ای توکن‌سازی خواهد شد که مدل BERT انجام می‌دهد.

چند نکته مهم در مورد این بخش در زیر آورده شده‌است:

- خروجی اولیه توکنایزر (مثلاً tokenizer(text)) معمولاً دیکشنری‌ای شامل لیست‌هایی از اعداد است، نه فقط یک لیست ساده، و مدل با این ساختار کار می‌کند.
- برای کار با Hugging Face datasets.Dataset، نیازی نیست که X و y را جدا کنیم .در واقع بهترین روش این است که:
  - دیتافریم یا Dataset شامل هر دو ستون باشد (مثلاً review و label)
  - سپس با استفاده از متد()map روی Dataset، فقط ستون review را توکنایز کنیم. (مدل بعداً از ستون‌های جدید استفاده خواهد کرد.)
- در مدل‌هایی مانند BERT، ابتدا متن به کمک توکنایزر به اعداد صحیح (توکن‌ها) تبدیل می‌شود. سپس این اعداد وارد embedding layer شده و به بردارهای معنایی چندبعدی تبدیل می‌شوند که به‌عنوان ورودی اصلی مدل برای یادگیری و پیش‌بینی استفاده می‌شوند.

توکنایزر مدل bert-base-cased را با دستور زیر بارگذاری کرده‌ایم.

```
# Tokenizer from a pretrained model
tokenizer = AutoTokenizer.from_pretrained("bert-base-cased")

# Take a look at the tokenizer
tokenizer
```

که خروجی آن به شکل زیر می‌باشد، و در ادامه در مورد آن توضیح خواهیم داد:

```
c:\Users\niush\anaconda3\lib\site-packages\huggingface_hub\file_download.py:896: FutureWarning: `resume_download` is deprecated
  warnings.warn(
```

tokenizer_config.json: 100% 49.0/49.0 [00:00<00:00, 3.06kB/s]

```
c:\Users\niush\anaconda3\lib\site-packages\huggingface_hub\file_download.py:144: UserWarning: `huggingface_hub` cache-system use
To support symlinks on Windows, you either need to activate Developer Mode or to run Python as an administrator. In order to act
  warnings.warn(message)
```

config.json: 100% 570/570 [00:00<00:00, 34.6kB/s]

vocab.txt: 100% 213k/213k [00:00<00:00, 644kB/s]

tokenizer.json: 100% 436k/436k [00:00<00:00, 1.30MB/s]

```
PreTrainedTokenizerFast(name_or_path='bert-base-cased', vocab_size=28996, model_max_len=512, is_fast=True, padding_side='right',
```

در این خروجی ۲ warning داده شده:

هشدار اول: resume_download به‌زودی حذف می‌شود چون دانلودها حالا به‌طور خودکار ادامه پیدا می‌کنند؛ اگر دانلود کامل می‌خواهی از force_download=True استفاده کن. ⇦ بی‌خطر و قابل چشم‌پوشی

هشدار دوم: سیستم شما از symlink پشتیبانی نمی‌کند، پس فایل‌ها به‌جای لینک‌شده، کامل ذخیره می‌شوند و ممکن است فضای بیشتری اشغال کنند.

⇦ بی‌خطر، فقط کمی فضای بیشتر مصرف می‌شود.

در ادامه، فایل‌های مربوط به این توکنایزر از سرور Hugging Face دانلود و در حافظه کش (cache) سیستم ذخیره شده‌اند.

فایل‌هایی که دانلود شده‌اند:

فایل	کاربرد
tokenizer_config.json	تنظیمات کلی توکنایزر (مثلاً اینکه fast tokenizer است یا نه، سمت padding، ...)
config.json	پیکربندی مدل BERT (مثلاً تعداد لایه‌ها، طول توکن ورودی، ...)
vocab.txt	فهرست کامل واژگان مدل (۲۸۰۹۹۶ توکن) که به هر توکن یک شناسه عددی می‌دهد.
tokenizer.json	ساختار دقیق و بهینه‌شده توکنایزر که برای عملکرد سریع‌تر استفاده می‌شود.

نهایت میبینیم که توکنایزر با ویژگی‌های روبرو بارگذاری شده‌است.

```
PreTrainedTokenizerFast(
    name_or_path='bert-base-cased',
    vocab_size=28996,
    model_max_len=512,
    is_fast=True,
    padding_side='right',
    truncation_side='right',
    special_tokens={
        'unk_token': '[UNK]',
        'sep_token': '[SEP]',
        'pad_token': '[PAD]',
        'cls_token': '[CLS]',
        'mask_token': '[MASK]'
    }
)
```

برای مدل BERT پنج توکن ویژه تعریف شده است. سایر مدل‌ها ممکن است توکن‌های ویژه متفاوتی داشته باشند:

- توکن‌هایی که در مجموعه‌داده آموزشی مدل BERT وجود نداشتند، به‌عنوان توکن ناشناخته در نظر گرفته می‌شوند. توکن ناشناخته [UNK] است و شناسه عددی آن 100 می‌باشد.
- توکن جداکننده [SEP] است و شناسه آن 102 می‌باشد.
- توکن پرکننده [PAD] (padding) است و شناسه آن 0 می‌باشد.
- توکن مربوط به طبقه‌بندی سطح جمله [CLS] است و شناسه آن 101 است.
- توکن ماسک‌شده [MASK] است و شناسه آن 103 می‌باشد.

در ادامه با استفاده از کد زیر ویژگی های ذکر شده که در توکنایزر هست را نیز چاپ می‌کنیم:

```
# Mapping between special tokens and their IDs.
print(f'The unknown token is {tokenizer.unk_token} and the ID for the unkown token is \
    {tokenizer.unk_token_id}.')
print(f'The seperator token is {tokenizer.sep_token} and the ID for the seperator token is \
    {tokenizer.sep_token_id}.')
print(f'The pad token is {tokenizer.pad_token} and the ID for the pad token is \
    {tokenizer.pad_token_id}.')
print(f'The sentence level classification token is {tokenizer.cls_token} and \
    the ID for the classification token is {tokenizer.cls_token_id}.')
print(f'The mask token is {tokenizer.mask_token} and the ID for the mask token is \
    {tokenizer.mask_token_id}.')
```

```
The unknown token is [UNK] and the ID for the unkown token is 100.
The seperator token is [SEP] and the ID for the seperator token is 102.
The pad token is [PAD] and the ID for the pad token is 0.
The sentence level classification token is [CLS] and the ID for the classification token is 101.
The mask token is [MASK] and the ID for the mask token is 103.
```

یک مثال از نوع خروجی تولید شده را نیز بررسی می‌کنیم، در مثال زیر جمله اول از دیتاست را برداشتیم و نوع خروجی تولید شده آن را بررسی کردیم:

```
# Get the first sentence from the dataset
sample_text = amz_review["review"].iloc[0]

# Apply the tokenizer to the sentence
tokenized_output = tokenizer(sample_text)

# Display the original text
print("Original Text:", sample_text)
print("\nTokenized Output:")

# Print each part of the tokenizer output on a separate line
for key, value in tokenized_output.items():
    print(f"{key}: {value}")
```

```
Original Text: So there is no way for me to plug it in here in the US unless I go by a converter.

Tokenized Output:
input_ids: [101, 1573, 1175, 1110, 1185, 1236, 1111, 1143, 1106, 15244, 1122, 1107, 1303, 1107, 1103,
token_type_ids: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
attention_mask: [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

- input_ids: اعداد متناظر با هر توکن در متن هستند این عددها بر اساس واژگان (vocabulary) مدل BERT تعیین می‌شوند. مدل یک دیکشنری از تمام توکن‌های ممکن دارد و به هر توکن یک عدد منحصر به‌فرد اختصاص داده شده‌است.

مدل BERT دارای یک واژگان ثابت از قبل آموزش‌دیده است که در زمان آموزش مدل ساخته شده و در فایل آن ذخیره شده است. که این مجموعه‌ای از توکن‌هاست (نه الزاماً کلمات کامل)، واژگان BERT-base معمولاً حدود 30,000 توکن دارد. برای اینکه مدل بتواند با کلمات ناآشنا هم کار کند. یا با الگوریتمی به نام WordPiece کلمات جدید را به بخش‌هایی که در واژگان هستند تقسیم می‌کند یا توکنایزر از [UNK] استفاده می‌کند. که بسیار نادر است. (مدل‌های جدیدتر مثل GPT-4 یا T5 از توکنایزرهای قوی‌تری استفاده می‌کنند مثل SentencePiece یا Byte-Pair Encoding که این مشکل را کمتر دارند.)

- token_type_ids: مشخص می‌کند هر توکن متعلق به جمله اول است یا دوم؛ در بیشتر موارد فقط شامل صفر است.
- attention_mask: نشان می‌دهد کدام توکن‌ها واقعی‌اند و مدل باید به آن‌ها توجه کند (1) و کدام‌ها padding هستند و باید نادیده گرفته شوند (0).

اگر به نمونه خروجی با دقت نگاه کنیم، میبینیم که اعداد 101 و 102 که در قسمت قبل توضیحات آنها داده شد واقعا در ابتدا و انتهای جملات وجود دارند و قرار گرفته اند.

```
input_ids: [101, 1573, 1175, 1110, 1185, 1236, 1111, 1143, 1106, 15244, 1122, 1107, 1303, 1107, 1103, 1646, 4895, 146, 1301, 1118, 170, 10454, 1200, 119, 102]
```

اگر بخواهیم همین توکن‌ها را به شکل کلمات قابل خواندن هم ببینیم، می‌توانیم از دستور زیر استفاده کنیم:

```
print("\nDecoded Tokens:")
print(tokenizer.convert_ids_to_tokens(tokenized_output["input_ids"]))
```

```
Decoded Tokens:
['[CLS]', 'So', 'there', 'is', 'no', 'way', 'for', 'me', 'to', 'plug', 'it', 'in', 'here', 'in', 'the',
```

در ادامه از ما خواسته شده که هنگام توکن‌سازی مجموعه نقدها با استفاده از توکنایزر، 3 پارامتر کلیدی را به‌طور مشخص تنظیم کنیم:

- max_length=32: طول هر ورودی حداکثر 32 توکن باشد؛ بیشتر از آن بریده و کمتر از آن با صفر پر می‌شود.
- truncation=True: برش توکن‌ها طبق max_length انجام می‌شود.
- padding='max_length': جمله‌های کوتاه با صفر تا طول 32 توکن پر می‌شوند.

این تنظیمات باعث می‌شوند همه ورودی‌ها هم‌اندازه و آماده ورود به مدل باشند. که برای اینکار با کد زیر ویژگی های مورد نظر را اعمال می‌کنیم:

```
def tokenize_dataset(data):
    return tokenizer(
        data["review"],
        padding="max_length",
        truncation=True,
        max_length=32
    )

dataset_train = hg_train_data.map(tokenize_dataset)
dataset_test = hg_test_data.map(tokenize_dataset)
```

✓ 0.1s

```
Map: 100% 800/800 [00:00<00:00, 6471.21 examples/s]
Map: 100% 200/200 [00:00<00:00, 6063.28 examples/s]
```

در ادامه با استفاده از print کردن آنها دیتای ساخته شده را بررسی می‌کنیم:

```
Dataset({
    features: ['review', 'label', '__index_level_0__', 'input_ids', 'token_type_ids', 'attention_mask'],
    num_rows: 800
})
Dataset({
    features: ['review', 'label', '__index_level_0__', 'input_ids', 'token_type_ids', 'attention_mask'],
    num_rows: 200
})
```

پس از توکن‌سازی، مشاهده می‌کنیم که هر دو Dataset مربوط به آموزش و آزمایش دارای ۶ ویژگی هستند: 'review'، 'label'، '__index_level_0__'، 'input_ids'، 'token_type_ids'، و 'attention_mask'. تعداد ردیف‌ها نیز با استفاده از ویژگی num_rows ذخیره می‌شود.

همچنین یک نمونه خروجی را چاپ می‌کنیم و آن را برای بررسی بهتر در فایل first_record.txt ذخیره می‌کنیم.

```
review: Thanks again to Amazon for having the things I need for a good price!
label: 1
__index_level_0__: 521
input_ids: [101, 5749, 1254, 1106, 9786, 1111, 1515, 1103, 1614, 146, 1444, 1111, 170, 1363, 3945, 106, 102,
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
token_type_ids: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
attention_mask: [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

dataset_train[0] محتوای اولین رکورد از دیتاست آموزشی را به‌صورت یک دیکشنری (ساختار کلید-مقدار) به ما می‌دهد.

- 'review' شامل متن نقد است. اولین نقد در دیتاست آموزشی این جمله است.
- 'label' برچسب مربوط به طبقه‌بندی را نشان می‌دهد. این رکورد یک نقد مثبت است، بنابراین مقدار برچسب آن 1 است.
- '__index_level_0__' اندیس رکورد در دیتافریم اصلی پانداس را نشان می‌دهد. عدد 521 یعنی رکورد اول در دیتاست آموزشی در واقع رکورد شماره 521 در دیتافریم اولیه بوده است.

Dataset حاصل به‌صورت خودکار **یک ستون جدید به نام** __index_level_0__ اضافه می‌کند که دقیقاً همان اندیس اصلی رکورد در دیتافریم پانداس است. چون اولین رکوردی که به طور تصادفی وارد train_data شده، در دیتافریم اصلی amz_review اندیسی برابر با 521 داشته، و این مقدار در ستون __index_level_0__ذخیره شده‌است.

- 'input_ids' شناسه‌های عددی توکن‌ها هستند. اینجا 22 شناسه وجود دارد چون max_length در فرآیند توکن‌سازی برابر با 32 تعیین شده است. و همانطور که دیده میشود پس از اندازه توکن های جمله ادامه آنها صفر گذاشته شده اند.
- 'token_type_ids' که به آن شناسه‌های بخش (segment IDs) هم گفته می‌شود:
  - مدل BERT برای دو وظیفه آموزش دیده است: Masked Language Modeling و Next Sentence Prediction.
  - token_type_ids در وظیفه دوم (تشخیص جمله بعدی) استفاده می‌شود که در آن دو جمله به مدل داده می‌شود تا تشخیص دهد آیا جمله دوم واقعاً دنباله جمله اول است یا نه.
  - در این حالت، توکن‌های جمله اول با صفر و جمله دوم با یک نمایش داده می‌شوند.
  - چون در مسئله طبقه‌بندی ما فقط تک جمله وجود دارد، تمام مقادیر token_type_ids برابر صفر هستند.
- 'attention_mask' مشخص می‌کند که مدل باید به کدام توکن‌ها توجه کند. توکن‌هایی که padding هستند مقدار صفر دارند، و توکن‌های واقعی مقدار یک.

6) Step 6: Load Pretrained Model

در این مرحله، با استفاده از انتقال یادگیری، یک لایه طبقه‌بندی (classification head) به انتهای مدل پیش‌آموخته BERT اضافه می‌کنیم. این head که شامل یک یا چند لایه Linear (Fully Connected) و معمولاً Softmax است، از ابتدا مقداردهی می‌شود و سپس با داده‌ی ما فاین‌تیون می‌شود. مدل BERT برای هر توکن بردار معنایی می‌سازد و بردار توکن [CLS] (که نماینده کل جمله است) به عنوان ورودی head استفاده می‌شود تا کلاس نهایی جمله (مثلاً مثبت یا منفی) پیش‌بینی شود.

انواع پیاده‌سازی مدل BERT برای طبقه‌بندی توالی:

دو روش رایج برای استفاده از مدل BERT جهت انجام تسک Sequence Classification وجود دارد که هرکدام مزایا و محدودیت‌های خاص خود را دارند:

۱. استفاده از مدل آماده‌ی Hugging Face:

```
model = AutoModelForSequenceClassification.from_pretrained("bert-base-cased", num_labels=2)
```

در این روش، مدل BERT به‌همراه یک head طبقه‌بندی آماده از کتابخانه Transformers بارگذاری می‌شود. مزیت اصلی آن سادگی، سرعت استفاده، هماهنگی با Trainer و ابزارهای ارزیابی Hugging Face است، اما انعطاف‌پذیری پایینی برای سفارشی‌سازی head دارد.

۲. ساختاردهی مدل سفارشی با PyTorch:

```
self.bert = BertModel.from_pretrained("bert-base-cased")
self.classifier = nn.Sequential(
    nn.Linear(768, 256),
    nn.ReLU(),
    nn.Linear(256, num_labels)
)
```

در این روش، تنها بدنه‌ی اصلی BERT بارگذاری شده و head **طبقه‌بندی توسط کاربر تعریف می‌شود**. این head می‌تواند شامل لایه‌های میانی، Dropout، فعال‌سازی‌ها یا سایر تنظیمات دلخواه به‌صورت سفارشی باشد. مزیت آن **انعطاف‌پذیری بالا در طراحی مدل** است، اما **کدنویسی پیچیده‌تر** بوده و برای استفاده با Trainer یا ارزیابی‌های خودکار به تنظیمات دستی نیاز دارد.

انتخاب حالت اول برای پیاده‌سازی:

```
# Load pretrained BERT model for sequence classification
from transformers import AutoModelForSequenceClassification

model = AutoModelForSequenceClassification.from_pretrained(
    "bert-base-cased",
    num_labels=2
)
```

✓ 4m 53.4s

```
c:\Users\niush\anaconda3\lib\site-packages\huggingface_hub\file_download.py:896: FutureWarning: `resume_download`
  warnings.warn(
Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP down
```

pytorch_model.bin: 100% 436M/436M [04:48<00:00, 1.48MB/s]

```
Some weights of the model checkpoint at bert-base-cased were not used when initializing BertForSequenceClassificat
- This IS expected if you are initializing BertForSequenceClassification from the checkpoint of a model trained on
- This IS NOT expected if you are initializing BertForSequenceClassification from the checkpoint of a model that y
Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-cased an
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
```

در نهایت با توجه به نیاز مساله از روش اول استفاده می‌کنیم. در کد بالا تابع AutoModelForSequenceClassification مدل BERT را بدون sequence classification head بارگذاری می‌کند.

متد ()from_pretrained وزن‌ها را از مدل پیش‌آموخته به مدل جدید انتقال می‌دهد، بنابراین وزن‌های مدل جدید به‌صورت تصادفی مقداردهی نمی‌شوند. اما توجه داشته باشید که وزن‌های بخش جدیدِ طبقه‌بندی توالی به‌صورت تصادفی مقداردهی می‌شوند.

bert-base-cased نام مدل پیش‌آموخته است که بسته به ماهیت پروژه، می‌توان آن را به مدل دیگری تغییر داد.

num_labels تعداد کلاس‌ها را مشخص می‌کند، چون دیتاست ما شامل دو کلاس (مثبت و منفی) است، پس مقدار num_labels=2 خواهد بود.

که در انتها با توجه به هشدار در خروجی مربوط به Xet Storage، که می‌گوید Hugging Face می‌تونه مدل‌ها رو با فناوری سریع‌تر Xet دانلود کنه و بدون نصب کتابخانه (hf_xet) به دانلود معمولی (HTTP) برمی‌گرده که سرعت پایین‌تری داره.، کتابخانه مورد نظر را نصب کردیم:

pip install hf_xet

7) Step 7: Set Training Argument

در **مرحله ۷**، آرگومان‌های مربوط به آموزش مدل را تنظیم می‌کنیم. در این مرحله، باید به مدل بگیم **چطوری آموزش ببینه** — یعنی تنظیمات مربوط به آموزش مثل تعداد epoch، اندازه batch، نحوه ذخیره‌سازی، گزارش‌گیری، و ارزیابی را مشخص می‌کنیم.

کتابخانه Hugging Face دارای **۹۶ پارامتر قابل تنظیم** برای کلاس TrainingArguments است که امکان شخصی‌سازی گسترده‌ای در فرآیند **فاین‌تیونینگ مدل‌های انتقال یادگیری** فراهم می‌کند.

نمونه‌هایی از انواع پارامترها مهم:

۱. مسیر ذخیره مدل و لاگ‌ها:

- output_dir: "sentiment_transfer_learning_transformer/"
- logging_dir: 'sentiment_transfer_learning_transformer/logs'

۲. نحوه گزارش‌گیری:

- logging_strategy: epoch (گزارش فقط در پایان هرepoch ، ساده و کافی برای داده کم)
- logging_steps: اگر 'strategy='steps باشد، هر چند گام یکبار گزارش بدهد.

۳. تنظیمات آموزش:

- num_train_epochs: ۴ (تعداد دفعاتی که داده را به مدل بدهیم، کافی برای فاین‌تیون روی دیتاست کوچک)
- per_device_train_batch_size: ۴ (سایز batch هنگام آموزش که برای سرعت و مصرف RAM مهم است.)
- learning_rate: 2e-5 (نرخ یادگیری ایمن برایBERT)

۴. نحوه ذخیره مدل در حین آموزش:

- save_strategy: epoch (ذخیره مدل در پایان هرepoch ، بدون تولید فایل زیاد)
- save_steps: اگر استراتژی ذخیره بر حسب steps باشد، فاصله بین ذخیره‌ها

۵. نحوه ارزیابی مدل:

- evaluation_strategy: epoch (ارزیابی عملکرد مدل در پایان هرepoch ، ساده و کافی)
- eval_steps: اگر استراتژی ارزیابی بر اساس step باشد، فاصله بین ارزیابی‌ها

۶. بارگذاری بهترین مدل در پایان آموزش:

- load_best_model_at_end: True (مدل بهترین نسخه خودش تضمین انتخاب بهترین مدل)
- **نکته مهم:** برای فعال‌سازی این گزینه، save_strategy و evaluation_strategy باید یکی باشند.

ما از متغیر های زیر استفاده کردیم:

```
# Set up training arguments
training_args = TrainingArguments(
    output_dir = "./sentiment_transfer_learning_transformer/",
    logging_dir = './sentiment_transfer_learning_transformer/logs',

    logging_strategy = 'epoch',
    logging_steps = 100, # not gonna be used, unless strategy be steps

    num_train_epochs = 4,

    per_device_train_batch_size = 4,
    per_device_eval_batch_size = 4,

    learning_rate = 2e-5,

    seed = 42,

    save_strategy = 'epoch',
    save_steps = 100,

    evaluation_strategy = 'epoch',
    eval_steps = 100,

    load_best_model_at_end = True,
    report_to = "none",  # Disables W&B logging
)
```

8) Step 8: Set Evaluation Metrics

در مرحله ۸، باید معیار ارزیابی (evaluation metric) را مشخص کنیم، چون کتابخانه Trainer در Hugging Face به‌صورت پیش‌فرض عملکرد مدل را حین آموزش به‌طور خودکار ارزیابی نمی‌کند.

Hugging Face کتابخانه‌ای به نام evaluate دارد که شامل بیش از ۱۰۰ ماژول ارزیابی است. برای مشاهده فهرست کامل این ماژول‌ها می‌توان از دستور evaluate.list_evaluation_modules() استفاده کرد.

```
# Number of evaluation modules
print(f'There are {len(evaluate.list_evaluation_modules())} evaluation models in Hugging Face.\n')

# List all evaluation metrics
evaluate.list_evaluation_modules()
```

✓ 4.2s

```
There are 207 evaluation models in Hugging Face.


['codeparrot/apps_metric',
 'lvwerra/test',
 'angelina-wang/directional_bias_amplification',
 'cpllab/syntaxgym',
```

در کل برای محاسبه متریک‌های ارزیابی مدل، می‌توان از دو کتابخانه‌ی اصلی استفاده کرد: sklearn.metrics و evaluate (کتابخانه رسمی Hugging Face)
کتابخانه sklearn انعطاف‌پذیری بیشتری دارد و برای محاسبه هم‌زمان چند متریک یا سفارشی‌سازی گزارش‌ها مناسب‌تر است، اما نیاز به کدنویسی بیشتر دارد. در مقابل، evaluate با Trainer کاملاً سازگار است، کد ساده‌تری دارد و به‌ویژه برای متریک‌های استاندارد مانند Accuracy و F1 بسیار سریع و راحت عمل می‌کند. در این پروژه، روش مبتنی بر evaluate انتخاب شده است تا یکپارچگی با سایر ابزارهای Hugging Face حفظ شود و پیاده‌سازی متریک‌ها ساده‌تر و خواناتر باشد.

متریک‌هایی که ما برای اجرا پیاده‌سازی کردیم:

متریک	کاربرد
Accuracy	درصد پیش‌بینی‌های درست از کل نمونه‌ها
Precision	نسبت پیش‌بینی‌های مثبت درست به کل پیش‌بینی‌های مثبت
Recall	نسبت پیش‌بینی‌های مثبت درست به کل موارد واقعی مثبت
F1 Score	میانگین هارمونیک precision و recall
AUC (ROC-AUC)	توانایی مدل در تشخیص مثبت از منفی مستقل از آستانه

بعد از محاسبه متریک‌ها، می‌توان به‌جای استفاده مستقیم از logits که داده های خام است، آن‌ها را با تابع softmax نرمال‌سازی کرد تا احتمال کلاس‌ها به دست آید؛ این کار برای متریک‌هایی مثل AUC ضروری است. تابع softmax را می‌توان با دو کتابخانه‌ی TensorFlow و PyTorch پیاده‌سازی کرد، اما از آنجایی که پروژه ما مبتنی بر PyTorch است، از F.softmax استفاده می‌کنیم که با ساختار Hugging Face Trainer سازگارتر است.

بنابراین، یک تابع سفارشی با نام compute_metrics تعریف کردیم تا بتوانیم عملکرد مدل طبقه‌بندی را با چند متریک مختلف ارزیابی کنیم. برای این منظور، از کتابخانه‌ی رسمی Hugging Face به نام evaluate استفاده کردیم که شامل ماژول‌های استاندارد محاسبه متریک است.

در نهایت، تمام مقادیر به‌صورت یک دیکشنری ساخت‌یافته بازگردانده می‌شوند تا هم در طول آموزش توسط Trainer نمایش داده شوند و هم برای گزارش‌گیری و مقایسه‌های بعدی قابل ذخیره و پردازش باشند.

```python
# Load evaluation metrics (this can be done once for efficiency)
accuracy_metric = evaluate.load("accuracy")
precision_metric = evaluate.load("precision")
recall_metric = evaluate.load("recall")
f1_metric = evaluate.load("f1")
roc_auc_metric = evaluate.load("roc_auc")  # For binary classification only

# Function to compute all metrics
def compute_metrics(eval_pred):
    logits, labels = eval_pred

    # Apply softmax to get normalized class probabilities
    probs = F.softmax(torch.tensor(logits), dim=-1).numpy()

    # Take the class with highest probability as final prediction
    preds = np.argmax(probs, axis=1)

    # Compute all metrics and return as a dictionary
    metrics = {
        "accuracy": accuracy_metric.compute(predictions=preds, references=labels)["accuracy"],
        "precision": precision_metric.compute(predictions=preds, references=labels)["precision"],
        "recall": recall_metric.compute(predictions=preds, references=labels)["recall"],
        "f1": f1_metric.compute(predictions=preds, references=labels)["f1"],
        "roc_auc": roc_auc_metric.compute(prediction_scores=probs[:, 1], references=labels)["roc_auc"]
    }

    return metrics
```

```
Downloading builder script: 100% 4.20k/4.20k [00:00<00:00, 227kB/s]
Downloading builder script: 100% 7.56k/7.56k [00:00<00:00, 33.2kB/s]
Downloading builder script: 100% 7.38k/7.38k [00:00<00:00, 852kB/s]
Downloading builder script: 100% 6.79k/6.79k [00:00<00:00, 625kB/s]
Downloading builder script: 100% 9.54k/9.54k [00:00<00:00, 1.17MB/s]
```

9) Step 9: Train Model Using Transformer Trainer

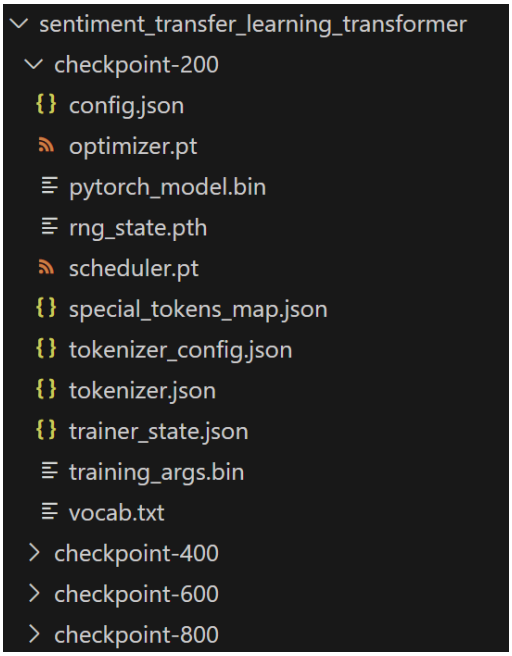
در مرحله ۹، مدل با استفاده از کلاس Trainer آموزش داده می‌شود. در این مرحله، مدل، آرگومان‌های آموزشی (TrainingArguments)، داده‌های آموزش و ارزیابی (در قالب Dataset) و تابع محاسبه متریک‌ها به Trainer داده می‌شوند. ستون‌هایی مثل review و __index_level_0__ که در فرآیند آموزش مورد استفاده مدل نیستند، به‌طور خودکار نادیده گرفته می‌شوند.

ستون __index_level_0__ فقط شماره ردیف دیتافریم است و اطلاعاتی برای مدل ندارد، و ستون review شامل متن خام است که در مرحله توکن‌سازی به ویژگی‌های عددی مانند input_ids تبدیل شده، بنابراین مدل مستقیماً با این دو ستون کاری ندارد و آن‌ها را نادیده می‌گیرد. در واقع مدل فقط از ویژگی‌های عددی‌شده‌ی نهایی استفاده می‌کند: token_type_ids, attention_mask, input_ids, (اختیاری) و label (برای یادگیری).

همچنین می‌توان از callbackهایی مثل EarlyStoppingCallback برای توقف زودهنگام استفاده کرد.

```python
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=dataset_train,
    eval_dataset=dataset_test,
    tokenizer=tokenizer,
    compute_metrics=compute_metrics,
    callbacks=[EarlyStoppingCallback(early_stopping_patience=1)]
)

trainer.train()
```



خروجی این آموزش فایل‌های زیر هستند که در ادامه آنها را توضیح می‌دهیم:

- *-checkpoint: شامل وضعیت کامل مدل در یک مرحله خاص از آموزش.
- pytorch_model.bin: وزن‌های مدل آموزش‌دیده برای بارگذاری مجدد.
- config.json: تنظیمات معماری مدل مانند تعداد لایه‌ها و اندازه‌های پنهان.
- training_args.bin: پارامترهای آموزشی مثل epoch و batch size.
- trainer_state.json: اطلاعات وضعیت آموزش، epoch و بهترین مدل.
- optimizer.pt & scheduler.pt: وضعیت به‌روز شده‌ی optimizer و scheduler برای ادامه آموزش از همان نقطه.
- rng_state.pth: وضعیت seed برای اطمینان از تکرارپذیری آموزش.
- توکنایزرها:
  - tokenizer.json: ساختار کامل توکنایزر.
  - tokenizer_config.json: تنظیمات توکنایزر (مثل padding، casing)
  - special_tokens_map.json: نقشه‌ی توکن‌های خاص مثل [CLS]
  - vocab.txt: لیست واژگان مورد استفاده در مدل.

نوع خروجی چاپ شده:

```
The following columns in the training set don't have a corresponding argument in `BertForSequenceClassif
***** Running training *****
  Num examples = 800
  Num Epochs = 4
  Instantaneous batch size per device = 4
  Total train batch size (w. parallel, distributed & accumulation) = 4
  Gradient Accumulation steps = 1
  Total optimization steps = 800
  Number of trainable parameters = 108311810
```




```
The following columns in the evaluation set don't have a corresponding argument in `BertForSequenceClass
***** Running Evaluation *****
  Num examples = 200
  Batch size = 4
{'loss': 0.0512, 'learning_rate': 1.5000000000000002e-05, 'epoch': 1.0}
```

سپس اطلاعات مهم مربوط به هر ecpoch را با کمک کد زیر در فایل checkpoint_metrics_summary.csv ذخیره کردیم.

```
base_dir = "sentiment_transfer_learning_transformer"

columns = [
    "checkpoint", "epoch", "step", "loss",
    "eval_accuracy", "eval_f1", "eval_loss",
    "eval_precision", "eval_recall", "eval_roc_auc",
    "eval_runtime"
]

rows = []

for subdir in os.listdir(base_dir):
    sub_path = os.path.join(base_dir, subdir)
    if os.path.isdir(sub_path):
        state_file = os.path.join(sub_path, "trainer_state.json")
        if os.path.exists(state_file):
            with open(state_file, "r") as f:
                trainer_state = json.load(f)
                logs = trainer_state.get("log_history", [])

                eval_logs = [log for log in logs if "epoch" in log and "eval_loss" in log]
                train_logs = [log for log in logs if "epoch" in log and "loss" in log]

                if eval_logs:
                    eval_log = eval_logs[-1]
                    step = eval_log.get("step")

                    matching_train = [log for log in train_logs if log.get("step") == step]
                    train_loss = matching_train[0]["loss"] if matching_train else None

                    row = {
                        "checkpoint": subdir,
                        "epoch": eval_log.get("epoch"),
                        "step": step,
                        "loss": train_loss,
                        "eval_accuracy": eval_log.get("eval_accuracy"),
                        "eval_f1": eval_log.get("eval_f1"),
                        "eval_loss": eval_log.get("eval_loss"),
                        "eval_precision": eval_log.get("eval_precision"),
                        "eval_recall": eval_log.get("eval_recall"),
                        "eval_roc_auc": eval_log.get("eval_roc_auc"),
                        "eval_runtime": eval_log.get("eval_runtime"),
                    }
                    rows.append(row)

df = pd.DataFrame(rows, columns=columns)
df.to_csv(os.path.join(base_dir, "checkpoint_metrics_summary.csv"), index=False)
```

و همینطور بهترین رکورد را هم ذخیره کردیم:

```
# Saving The Best model

trainer.save_model("best_model_bert_sentiment")
tokenizer.save_pretrained("best_model_bert_sentiment")
```

✓ 3.7s

```
Saving model checkpoint to best_model_bert_sentiment
Configuration saved in best_model_bert_sentiment\config.json
Model weights saved in best_model_bert_sentiment\pytorch_model.bin
tokenizer config file saved in best_model_bert_sentiment\tokenizer_config.json
Special tokens file saved in best_model_bert_sentiment\special_tokens_map.json
tokenizer config file saved in best_model_bert_sentiment\tokenizer_config.json
Special tokens file saved in best_model_bert_sentiment\special_tokens_map.json

('best_model_bert_sentiment\\tokenizer_config.json',
 'best_model_bert_sentiment\\special_tokens_map.json',
 'best_model_bert_sentiment\\vocab.txt',
 'best_model_bert_sentiment\\added_tokens.json',
 'best_model_bert_sentiment\\tokenizer.json')
```

خروجی فایل csv:

	checkpoint	epoch	step	loss	eval_accuracy	eval_f1	eval_loss	eval_precision	eval_recall	eval_roc_auc	eval_runtime
0	checkpoint-200	1.0	200	0.0512	0.935	0.934673	0.613324	0.920792	0.948980	0.970988	2.6867
1	checkpoint-400	2.0	400	0.0421	0.930	0.927083	0.662974	0.946809	0.908163	0.975590	3.7534
2	checkpoint-600	3.0	600	0.0077	0.930	0.927835	0.747440	0.937500	0.918367	0.969838	4.5610
3	checkpoint-800	4.0	800	0.0000	0.925	0.923077	0.791337	0.927835	0.918367	0.973489	4.4384

تحلیل نتایج:

1. Train Loss (loss):

- مقدار train loss به شکل یکنواخت کاهش پیدا کرده تا جایی که در epoch 4 تقریباً صفر شده.
- اما این افت زیاد در loss آموزشی می‌تونه نشونه‌ی overfitting باشه، چون همزمان eval_loss داره بالا می‌ره.

2. Eval Accuracy:

- در epoch 1 بالاترین مقدار رو داریم: 93.5%
- بعد از اون به تدریج افت کرده تا 92.5% در epoch 4
- این نشون می‌ده که مدل در حال **از دست دادن تعمیم‌پذیری** هست.

3. Eval Loss:

- از ۰٫۶۱ در epoch 1 رسیده به ۰٫۷۹ در epoch 4 → یعنی **عملکرد مدل روی داده‌ی تست بدتر شده.**

4. F1 Score:

- افت کم اما پیوسته داره (از ۰٫۹۳۴ به ۰٫۹۲۳) ⟶ نشان‌دهنده کاهش کلی عملکرد.

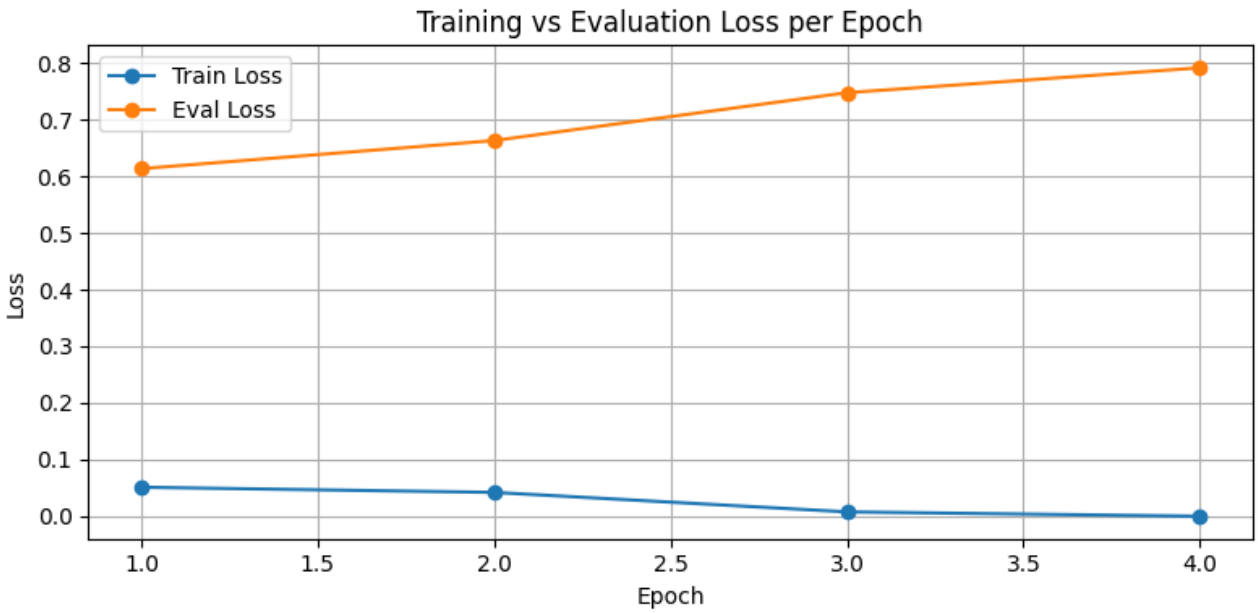
5. Precision vs Recall:

- دقت (precision) در epoch 2 بالاست، ولی Recall در epoch 1 بهتره.
- این یعنی در epoch 1 مدل بهتر تونسته موارد مثبت رو **شناسایی کنه** (high recall)، ولی در epoch 2 بهتر تونسته **اشتباه مثبت نده** (high precision).

6. AUC (ROC-AUC):

- بیشترین مقدار در epoch 2: 0.9756
- با این حال، اختلاف AUC زیاد نیست و مدل در تمام مراحل توان خوبی در تفکیک مثبت/منفی داشته.

سپس نمودار loss برای داده‌های train و evaluation را رسم کردیم:



از لحاظ زمانی این مرحله حدود ۹ دقیقه طول کشیده‌است.

علت ضعف مدل:

۱. اندازه بسیار محدود داده‌های آموزشی: مدل BERT برای عملکرد مطلوب نیاز به تعداد زیادی نمونه دارد. در این پروژه، تنها ۸۰۰ نمونه موجود بود (حدود ۶۴۰ برای آموزش و ۱۶۰ برای ارزیابی)، که برای fine-tuning یک مدل بزرگ مانند BERT ناکافی است.
۲. پتانسیل بالا برای Overfitting: با توجه به حجم پایین داده، مدل به سرعت به داده‌های آموزشی مسلط شده (train loss نزدیک به صفر)، در حالی که عملکرد روی داده‌های اعتبارسنجی کاهش یافته (eval loss رو به افزایش)، که نشانه‌ی overfitting است.
۳. کوچک بودن Batch Size: استفاده از batch size برابر با می‌تواند باعث روند یادگیری را ناپایدار کند.
۴. عدم استفاده از روش‌های کاهش overfitting: در این آموزش، از تکنیک‌هایی مانند weight decay، dropout اضافه، یا early stopping با معیار مناسب استفاده نشده است که می‌توانستند به کنترل بیش‌برازش کمک کنند.
۵. ماهیت ساده و عمومی داده‌ها: دیتاست Amazon Cell Labelled شامل نظراتی نسبتاً کوتاه و بدون ساختار پیچیده است. مدل ممکن است در درک دقیق احساسات در چنین متن‌هایی دچار ابهام شود.
۶. عدم بهره‌گیری از افزایش داده (Data Augmentation): روش‌هایی مانند synonym replacement، back-translation یا paraphrasing می‌توانستند در افزایش حجم داده و تنوع ساختار زبانی کمک‌کننده باشند.

بهینه‌سازی نتایج با روش بهبود متغیرها:

ابتدا با توجه به نتایج مرحله قبل همچنان بصورت دستی برخی از پارامترها را برای بهبود نتایج و کاهش overfitting تغییر دادیم:

```
training_args_var_improved = TrainingArguments(
    output_dir = "./sentiment_transfer_learning_transformer_improved/",
    logging_dir = './sentiment_transfer_learning_transformer_improved/logs',

    logging_strategy = 'epoch',
    evaluation_strategy = 'epoch',
    save_strategy = 'epoch',
    load_best_model_at_end = True,

    num_train_epochs = 10,
    per_device_train_batch_size = 8,
    per_device_eval_batch_size = 8,

    learning_rate = 1e-5,
    weight_decay = 0.01,
    lr_scheduler_type = "linear",
    warmup_steps = 50,

    seed = 42,
    report_to = "none",
)
```

۱. **افزایش تعداد epochها از 4 به 10:**
این تغییر با هدف دادن زمان بیشتر به مدل برای یادگیری بهتر انجام شد، به‌خصوص با در نظر گرفتن استفاده از مکانیزم‌هایی مثل weight decay و scheduler که جلوی overfitting را می‌گیرند.

۲. **افزایش batch size از 4 به 8 برای آموزش و ارزیابی**
با افزایش batch size، گرادیان‌ها پایدارتر شده و مدل در هر به‌روزرسانی اطلاعات بیشتری دریافت می‌کند. این موضوع می‌تواند در بهبود همگرایی مؤثر باشد.

۳. **کاهش نرخ یادگیری از 2e-5 به 1e-5**
این کاهش باعث می‌شود مدل آهسته‌تر و دقیق‌تر یاد بگیرد، که برای شرایطی با داده‌ی کم یا نویزی مناسب‌تر است و می‌تواند جلوی نوسان یا یادگیری نادرست در مراحل اولیه را بگیرد.

۴. **افزودن weight decay برابر با 0.01**
این پارامتر به عنوان یک تکنیک regularization استفاده می‌شود و باعث می‌شود مدل وزن‌های غیرضروری را کمتر یاد بگیرد؛ در نتیجه احتمال overfitting کاهش می‌یابد.

۵. **استفاده از lr_scheduler از نوع linear**
این نوع scheduler باعث کاهش تدریجی نرخ یادگیری در طول آموزش می‌شود که برای مدل‌های ترنسفورمر، از جمله BERT، پیشنهاد شده و به پایداری آموزش کمک می‌کند.

۶. **افزودن warmup_steps برابر با 50**
warmup باعث می‌شود در ابتدای آموزش، نرخ یادگیری به‌صورت تدریجی افزایش یابد که باعث جلوگیری از ناپایداری گرادیان‌ها در مراحل ابتدایی می‌شود.

	checkpoint	epoch	step	loss	eval_accuracy	eval_f1	eval_loss	eval_precision	eval_recall	eval_roc_auc	eval_runtime
0	checkpoint-100	1.0	100	0.0000	0.930	0.927083	0.884364	0.946809	0.908163	0.970688	7.4232
1	checkpoint-1000	10.0	1000	0.0000	0.930	0.927083	0.738939	0.946809	0.908163	0.979242	7.0179
2	checkpoint-200	2.0	200	0.0334	0.930	0.926316	0.808202	0.956522	0.897959	0.971589	6.9289
3	checkpoint-300	3.0	300	0.0167	0.930	0.927835	0.704374	0.937500	0.918367	0.975440	6.7717
4	checkpoint-400	4.0	400	0.0102	0.920	0.918367	0.734448	0.918367	0.918367	0.975890	6.8681
5	checkpoint-500	5.0	500	0.0000	0.915	0.913706	0.748440	0.909091	0.918367	0.975840	6.7318
6	checkpoint-600	6.0	600	0.0111	0.930	0.926316	0.832469	0.956522	0.897959	0.979242	6.7292
7	checkpoint-700	7.0	700	0.0078	0.930	0.927835	0.720854	0.937500	0.918367	0.977741	6.7157
8	checkpoint-800	8.0	800	0.0001	0.935	0.933333	0.710412	0.938144	0.928571	0.977641	6.7335
9	checkpoint-900	9.0	900	0.0000	0.930	0.927083	0.738305	0.946809	0.908163	0.979242	6.7353

تحلیل نتایج:

1. Train Loss (ستون loss)

- در اکثر epochها، مقدار loss به‌سرعت به نزدیک صفر رسیده.
- این نشون می‌ده که مدل **کاملاً به داده‌های آموزش مسلط شده** و حتی احتمالاً overfit کرده.

2. Eval Loss (ستون eval_loss)

- مقدار eval loss از 0.88~در epoch 1 کاهش پیدا کرده و در epoch 8 به کمترین مقدار (0.7104) رسیده.
- در epoch 9 و 10 کمی بالا رفته (0.7383).
- این یعنی عملکرد مدل روی داده‌های validation در epoch 8 بهترین بوده.
- نسبت به متغیرهای حالت قبلی بهبود داشتیم.

3. Eval Accuracy

- نسبتاً **ثابت** بوده: بین 0.035 – 0.0915
- **بیشترین دقت** در epoch 8

4. Eval F1 Score

- F1 در epoch 8 به **بیشترین مقدار** 0.9333 رسیده
- به‌وضوح بهتر از epochهای اول

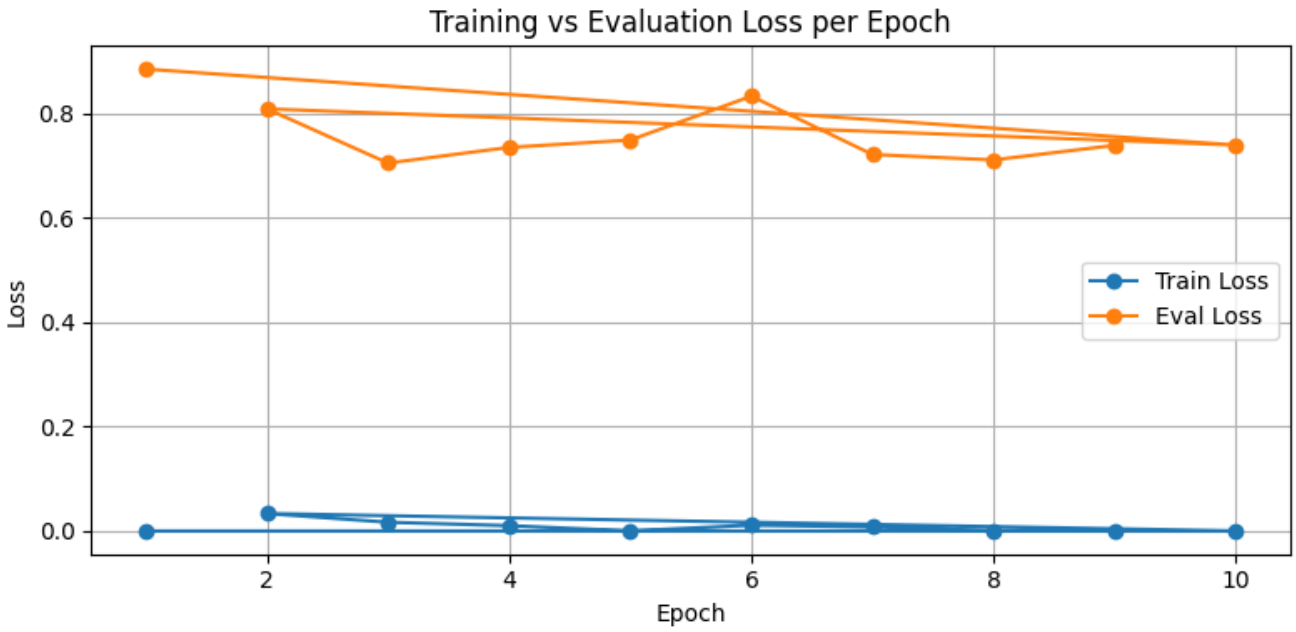
5. Precision و Recall

- Precision در همه مراحل بالا و پایدار بوده (91%-95%)
- Recall هم وضعیت خوبی داشته، ولی در epoch 8 بالاترین مقدارش رو داشته (0.9286)

6. ROC-AUC

- همه مقادیر بالاتر از 0.97 بودن که نشون‌دهنده‌ی **توانایی قوی مدل در تمایز کلاس‌ها**ست.
- بهترین مقدار ROC-AUC هم در epoch 10 با مقدار 0.9792 بوده اما تفاوت با بقیه خیلی زیاد نیست.

سپس نمودار loss برای داده‌های train و evaluation را رسم کردیم:



از لحاظ زمانی این مرحله حدود ۴۳ دقیقه طول کشیده‌است.

بهینه‌سازی نتایج با الگوریتم بهینه‌سازی Optuna ، یکی از روش‌های Hyper Parameters Tuning:

برای بهینه‌سازی عملکرد مدل، نیاز به تنظیم دقیق ابرپارامترها (hyperparameters) داشتیم. در ابتدا این کار به‌صورت دستی انجام می‌شد، اما به‌دلیل محدودیت‌های این روش، از Optuna به‌عنوان یکی از روش‌های مؤثر در hyperparameter tuning استفاده کردیم. Optuna با جستجوی هوشمند بین ترکیب‌های مختلف پارامترها مانند learning rate، batch size و تعداد epoch، بهترین تنظیمات را به‌صورت خودکار پیدا می‌کند. این رویکرد باعث افزایش دقت، کاهش خطای ارزیابی و صرفه‌جویی در زمان شد و به‌صورت یکپارچه با Trainer در کتابخانه Transformers قابل استفاده بود.

۱. مرحله ۱: نصب Optuna و setup اولیه

pip install optuna

۲. مرحله ۲: ابتدا تابع مدل را تعریف می‌کنیم:
برای اینکه Trainer بتونه چندبار مدل جدید بسازه (برای تست پارامترهای مختلف)، باید این تابع رو براش تعریف کنیم.

```
def model_init():
    return AutoModelForSequenceClassification.from_pretrained("bert-base-cased", num_labels=2)
```

۳. مرحله ۳: متغیرهای ورودی را تعریف می‌کنیم:
که در آن مشخص می‌کنیم که چه ترکیب‌هایی از learning_rate, weight_decay, batch_size و ... مدل رو آموزش بدهد، و برگرداند eval_loss یا eval_f1. در این مرحله، مشخص می‌کنیم که Optuna باید روی کد ما چه پارامترهایی رو تغییر بده و با چه هدفی (مثلاً کمینه کردن eval_loss) مدل رو آموزش بده.

```
# Define basic training arguments (no model saving/logging to avoid clutter)
training_args_hyperparam_tune = TrainingArguments(
    output_dir="./sentiment_transfer_learning_transformer_hyperparam_tune/",
    evaluation_strategy="epoch",         # Evaluate at the end of each epoch
    save_strategy="no",                  # Don't save checkpoints during tuning
    logging_strategy="epoch",            # Log metrics per epoch
    report_to="none",                    # Disable W&B or other integrations
    disable_tqdm=True,                   # Clean output (optional)
    load_best_model_at_end=False         # Not needed for hyperparameter search
)
```

۴. مرحله ۴: در این مرحله هایپرپارامترهای ورودی مدل را تعریف می‌کنیم:

```
# Create a Trainer object (note: model is created via model_init)
trainer_hyperparam_tune = Trainer(
    model_init=model_init,              # Function to reinitialize model for each trial
    args=training_args_hyperparam_tune,
    train_dataset=dataset_train,
    eval_dataset=dataset_test,
    compute_metrics=compute_metrics     # Your custom metrics (e.g. F1, accuracy)
)
```

5. مرحله ۵: بخش tuning را با تابع آماده Trainer.hyperparameter_search اجرا می‌کنیم:

```
# Perform hyperparameter search using Optuna
best_run_hyperparam_tune = trainer_hyperparam_tune.hyperparameter_search(
    direction="maximize",               # Optimize for higher F1 score
    backend="optuna",                   # Use Optuna as the optimization engine
    n_trials=10,                        # Number of different combinations to test
    hp_space=lambda trial: {
        "learning_rate": trial.suggest_float("learning_rate", 1e-5, 5e-5, log=True),
        "num_train_epochs": trial.suggest_int("num_train_epochs", 2, 5),
        "per_device_train_batch_size": trial.suggest_categorical("per_device_train_batch_size", [4, 8]),
        "weight_decay": trial.suggest_float("weight_decay", 0.0, 0.3),
    }
)

# Print the best set of hyperparameters found
print("Best run:")
print(best_run_hyperparam_tune)
```

از لحاظ زمانی این مرحله حدود ۷۰ دقیقه طول کشیده‌است.

۶. سپس نتایج را برای بررسی بهتر در فایل csv دخیره می‌کنیم.

```python
# Access the full Optuna study object from best_run
study = best_run_hyperparam_tune.trial._study

# Extract all trials
trials = study.trials

# Build a list of trial results
rows = []
for trial in trials:
    row = {
        "trial_number": trial.number,
        "status": trial.state.name,
        "objective_value": trial.value,
        **trial.params  # Unpack all hyperparameters (learning_rate, batch_size, etc.)
    }
    rows.append(row)

# Create DataFrame and save to CSV
df_trials = pd.DataFrame(rows)
df_trials.to_csv("optuna_hyperparameter_trials.csv", index=False)

print("All trial results saved to optuna_hyperparameter_trials.csv")
```

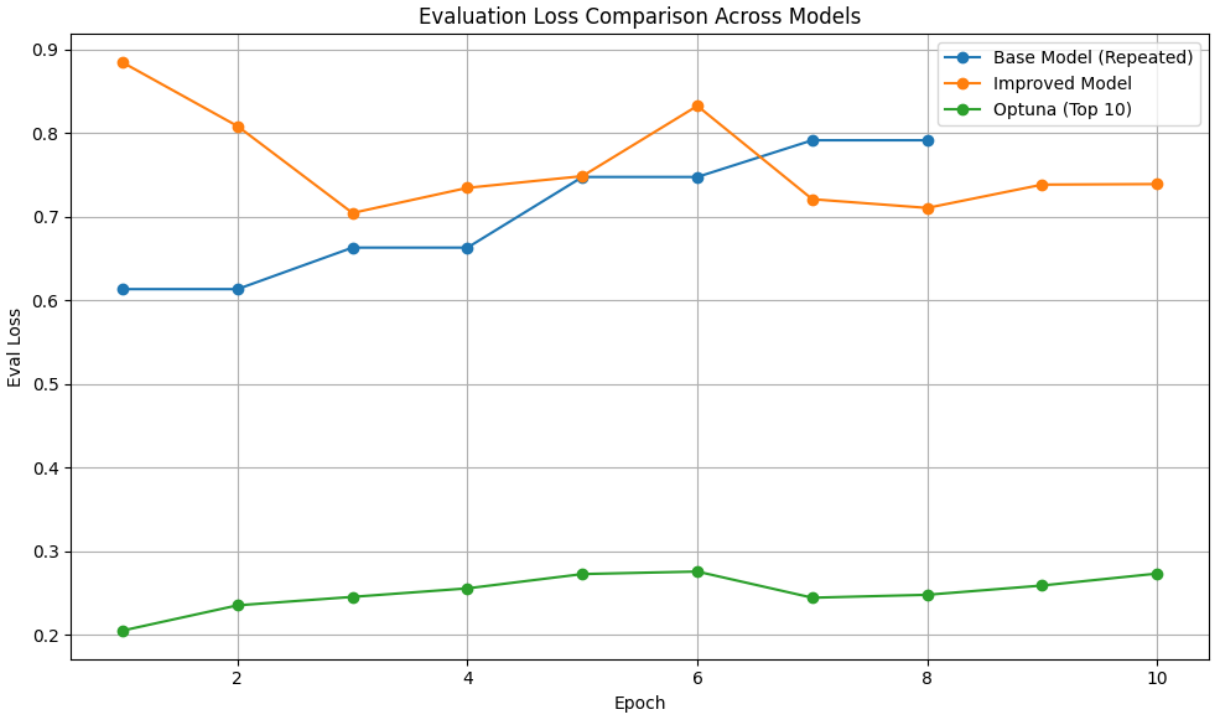
همچنین یک کپی از کل جزییات نتایج در فایل optuna_log_output.txt قرار می‌دهیم

نتایج رو بصورت sort شده از نظر evaluation_loss اعمال کردیم:

trial_numb	learning_ra	num_train_	per_device_	weight_dec	eval_loss	eval_accura	eval_precis	eval_recall	eval_f1	eval_roc_a	eval_runtin	eval_sampl	eval_steps_	epoch
1	3.54E-05	2	4	0.271171	0.412	0.9	0.875	0.9286	0.901	0.938	3.1376	63.744	7.968	1
2	3.90E-05	5	8	0.106781	0.2962	0.95	0.9681	0.9286	0.9479	0.9788	7.0651	28.308	3.538	2
2	3.90E-05	5	8	0.106781	0.3372	0.905	0.8835	0.9286	0.9055	0.9403	34.052	5.873	0.734	1
2	3.90E-05	5	8	0.106781	0.3101	0.925	0.9368	0.9082	0.9223	0.9776	33.8045	5.916	0.74	2
2	3.90E-05	5	8	0.106781	0.3741	0.935	0.9474	0.9184	0.9326	0.9724	33.7928	5.918	0.74	3
2	3.90E-05	5	8	0.106781	0.4839	0.925	0.9278	0.9184	0.9231	0.9727	33.7848	5.92	0.74	4
3	2.20E-05	3	4	0.214134	0.4813	0.93	0.9375	0.9184	0.9278	0.9747	9.2792	21.554	2.694	5
3	2.20E-05	3	4	0.214134	0.2756	0.93	0.9468	0.9082	0.9271	0.9611	3.0538	65.492	8.187	1
3	2.20E-05	3	4	0.214134	0.3269	0.93	0.9565	0.898	0.9263	0.9783	3.3021	60.568	7.571	2
4	4.99E-05	5	8	0.279752	0.3286	0.935	0.957	0.9082	0.9319	0.9829	3.332	60.024	7.503	3
4	4.99E-05	5	8	0.279752	0.205	0.94	0.9057	0.9796	0.9412	0.9766	9.1282	21.91	2.739	1
4	4.99E-05	5	8	0.279752	0.2443	0.935	0.9775	0.8878	0.9305	0.9868	8.976	22.282	2.785	2
4	4.99E-05	5	8	0.279752	0.3577	0.94	0.9479	0.9286	0.9381	0.9797	8.5968	23.265	2.908	3
4	4.99E-05	5	8	0.279752	0.352	0.94	0.9574	0.9184	0.9375	0.9808	8.7833	22.77	2.846	4
5	1.27E-05	4	8	0.256597	0.382	0.94	0.9574	0.9184	0.9375	0.9804	8.6552	23.108	2.888	5
5	1.27E-05	4	8	0.256597	0.2726	0.93	0.9375	0.9184	0.9278	0.9686	5.6861	35.174	4.397	1
5	1.27E-05	4	8	0.256597	0.2478	0.93	0.9667	0.8878	0.9255	0.9731	6.5229	30.661	3.833	2
5	1.27E-05	4	8	0.256597	0.2589	0.95	0.9583	0.9388	0.9485	0.9738	5.5656	35.935	4.492	3
6	4.66E-05	5	8	0.250535	0.2732	0.94	0.9479	0.9286	0.9381	0.9761	4.385	45.61	5.701	4
7	3.23E-05	3	4	0.208944	0.2352	0.915	0.8785	0.9592	0.9171	0.9723	8.5315	23.442	2.93	1
8	3.51E-05	4	8	0.229363	0.3227	0.925	0.9109	0.9388	0.9246	0.9433	10.0135	19.973	2.497	1
9	3.63E-05	5	8	0.224975	0.2807	0.905	0.9647	0.8367	0.8962	0.9589	8.3877	23.844	2.981	1
10	2.15E-05	5	8	0.169761	0.2555	0.925	0.9278	0.9184	0.9231	0.964	7.2725	27.501	3.438	1
10	2.15E-05	5	8	0.169761	0.2453	0.925	0.9462	0.898	0.9215	0.9654	8.1834	24.44	3.055	1

همچنان overfitting دیده میشه، اما نتایج بصورت قابل توجهی بهبود پیدا کردند، و همانطور که دیده میشه، نتیجه هایلایت شده، بهترین مقادیر رو داشته.

در نهایت برای مقایسه ۳ روش اعمال شده، آنها را با نمودار زیر مقایسه می‌کنیم:



تحلیل نمودار Evaluation Loss:

- Base Model (Repeated)
  - با تکرار هر مقدار برای رسیدن به ۱۰ نقطه رسم شده.
  - روند نسبتاً **صعودی** داره، اما مدل با ادامه‌ی آموزش، **overfit** کرده یا نتونسته بیشتر بهبود بده.
  - eval_loss بین ~۰٫۶۲ تا ~۰٫۷۹ نوسان داشته.
  - **نتیجه:** مدل اولیه از نظر بهینه‌سازی خیلی جای پیشرفت داشته و نتیجه ضعیفی داشته.
- Improved Model

- نسخه‌ی بهبود یافته با:
  - epoch=10
  - batch_size=8
  - learning_rate=1e-5
  - weight_decay و warmup اضافه شده
- در ابتدا کاهش قابل توجه eval_loss رو نشون می‌ده (از ~۰٫۸۸ به ~۰٫۷۰ در epoch 3)
- اما بعدش **نوسانات زیادی** داره (مثلاً در epoch 6 تا ~۰٫۸۳ افزایش پیدا کرده)
- **نتیجه:** تنظیمات بهتر شده، ولی هنوز پایدار نیست، نیاز به fine-tune بیشتر یا early stopping داره.

➢ Optuna (Top 10)

- بهترین ۱۰ نتیجه‌ی حاصل از ترکیب‌های مختلف پارامترهاست.
- eval_loss در همه نقاط **به‌مراتب پایین‌تر** از دو مدل قبلیه.
- مقدار eval_loss در محدوده‌ی 0.20 تا **0.28** مونده، یعنی خیلی پایدار و خوب
- روند کلی: کم‌کم افزایش، ولی همچنان در سطح بسیار پایین باقی مونده.
- **نتیجه:**
  - Optuna به طرز محسوسی بهتر عمل کرده
  - بهترین ترکیب‌ها رو با دقت بالا پیدا کرده
  - هم **کیفیت بالاتر** داشته هم **پایداری بهتر**

نتیجه‌گیری نهایی:

مدل	کیفیت نهایی(Eval Loss)	روند تغییر	نتیجه
Base	نسبتاً بالا (~0.79)	افزایشی/نوسانی	ضعیف‌ترین عملکرد
Improved	نسبتاً متوسط (~0.70–0.83)	نوسانی	قابل قبول، اما ناپایدار
Optuna (Top 10)	بسیار پایین (~0.21–0.28)	پایدار و منظم	**بهترین عملکرد و پایداری**

10) Step 10: Make Predictions for Text Classification

خلاصه مراحل پیش‌بینی و استخراج نتایج این بخش:

۱) **اجرای مدل روی داده تست**
۲) **استخراج** logits
۳) **تبدیل** logits **به احتمال**
۴) **تعیین برچسب نهایی**
۵) **استخراج برچسب‌های واقعی**

⇦ **مرحله ۱:** اجرای مدل روی داده تست

با استفاده از متد ( ) predict . از کلاس Trainer، پیش‌بینی روی داده تست را اجرا می‌کنیم.

خروجی آن یک شیء از نوع PredictionOutput است که شامل سه چیز است:

- . predictions → logits array
- . label_ids → actual labels
- . metrics → accuracy, loss, time, …

```
# Run predictions using the fine-tuned model
y_test_predict = trainer_hyperparam_tune.predict(dataset_test)

y_test_predict
```

✓ 15.4s

```
The following columns in the test set don't have a corresponding argument in `BertForSequenceClassification.forward`
***** Running Prediction *****
  Num examples = 200
  Batch size = 8
```

که خروجی آن نیز به شکل زیر است:

```
PredictionOutput(predictions=array([[-2.1604018 ,  1.830906  ],
       [-2.2700777 ,  1.8400943 ],
       [-2.2391727 ,  1.9554585 ],
       [ 1.3721502 , -1.3700819 ],
       [ 1.1361887 , -1.1619025 ],
       [-2.407266  ,  1.9388683 ],
       [ 1.007642  , -1.0536374 ],
       [ 1.4199715 , -1.3783813 ],
       [-2.0942442 ,  1.7481089 ],
       [ 1.3480712 , -1.3903517 ],
       [-2.2798731 ,  1.9994271 ],
       [ 0.98099893, -1.1100652 ],
       [-0.77576333,  1.1260519 ],
       [-2.5000134 ,  2.0330675 ],
       [-2.3965197 ,  2.129101  ],
       [ 0.8080169 , -0.850708  ],
       [ 1.1049882 , -1.1110027 ],
       [-2.497545  ,  2.050708  ],
       [-2.4694383 ,  1.9792547 ],
       [ 1.4047484 , -1.4039563 ],
       [-2.2587724 ,  1.9711275 ],
       [ 0.9856377 , -1.0036421 ],
       [-2.4913738 ,  2.1409335 ],
       [ 1.3419591 , -1.2719984 ],
       [-1.6166834 ,  1.6044862 ],
...
       0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1,
       0, 0, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1, 0, 0,
       0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0,
       0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 0,
       0, 0], dtype=int64), metrics={'test_loss': 0.24525555968284607, 'test_accuracy': 0.925, 'test_precision': 0.946236559139785,
```

⇦ **مرحله ۲:** استخراج logits

در این اینجا، بخش اول خروجی قسمت قبل را که مربوط به logitها هست را می‌خواهیم استخراج کنیم، اینها داده‌های خام برای هر نمونه قبل از اعمال softmax هستند، این خروجی ها پیش‌بینی‌های ماست، که بعدا با لیبل‌های واقعی که در قسمت دوم خروجی قبل بود، مقایسه خواهند شد، و همینطور ۵ نمونه از خروجی های آن را نیز چاپ کردیم:

```
# Predicted logits
y_test_logits = y_test_predict.predictions

# First 5 predicted probabilities
y_test_logits[:5]
```

✓ 0.0s

```
array([[-2.1604018,  1.830906 ],
       [-2.2700777,  1.8400943],
       [-2.2391727,  1.9554585],
       [ 1.3721502, -1.3700819],
       [ 1.1361887, -1.1619025]], dtype=float32)
```

⇦ **مرحله ۳:** تبدیل logits به احتمال

با استفاده از تابع softmax، logitها را به احتمال برای هر کلاس تبدیل می‌کنیم، به‌طوری که مجموع احتمال‌ها برای هر نمونه برابر با ۱ باشد.

سپس با numpy() . از tensor به آرایه Numpy تبدیل می‌کنیم.

```
y_test_probabilities = F.softmax(torch.tensor(y_test_logits), dim=1).numpy()

y_test_probabilities[:5]
```

✓ 0.0s

```
array([[0.01814038, 0.9818596 ],
       [0.01614017, 0.9838598 ],
       [0.01485238, 0.98514766],
       [0.9394731 , 0.06052686],
       [0.9087188 , 0.09128115]], dtype=float32)
```

⇦ **مرحله ۴:** تعیین برچسب نهایی

با اعمال argmax روی احتمال هر نمونه، برچسب نهایی (۰ یا ۱) را با توجه به احتمال بزرگ‌تر انتخاب می‌کنیم. نتیجه، یک آرایه‌ی یک‌بعدی شامل برچسب‌های پیش‌بینی‌شده است. و همینطور ۵ نمونه از خروجی را هم چاپ کردیم:

```
# Predicted labels
y_test_pred_labels = np.argmax(y_test_probabilities, axis=1)

# First 5 predicted probabilities
y_test_pred_labels[:5]
```

✓ 0.0s

```
array([1, 1, 1, 0, 0], dtype=int64)
```

⇦ **مرحله ۵: استخراج برچسب‌های واقعی**

برچسب‌های صحیح نمونه‌ها از همان خروجی اولیه‌ی مدل (label_ids) را استخراج کرده و برای ارزیابی مدل استفاده می‌کنیم.

```
# Actual labels
y_test_actual_labels = y_test_predict.label_ids

# First 5 predicted probabilities
y_test_actual_labels[:5]
```

✓ 0.0s

```
array([1, 1, 1, 0, 0], dtype=int64)
```

11) Step 11: Model Performance Evaluation

در مرحله ۱۱، عملکرد مدل دسته‌بندی متنی مبتنی بر یادگیری انتقالی را ارزیابی خواهیم کرد. استفاده از trainer.evaluate یک روش سریع برای به‌دست آوردن مقدار loss و دقت (accuracy) روی دیتاست تست است.

که در واقع روی بهترین متغیرهای مدل اعمال می‌شود.

```
# Trainer evaluate
trainer_hyperparam_tune.evaluate(dataset_test)
```

✓ 15.1s

```
The following columns in the evaluation set don't have a corresponding argument in `BertForSeque
***** Running Evaluation *****
  Num examples = 200
  Batch size = 8
{'eval_loss': 0.24525555968284607, 'eval_accuracy': 0.925, 'eval_precision': 0.946236559139785,
```

```
{'eval_loss': 0.24525555968284607,
 'eval_accuracy': 0.925,
 'eval_precision': 0.946236559139785,
 'eval_recall': 0.8979591836734694,
 'eval_f1': 0.9214659685863874,
 'eval_roc_auc': 0.9653861544617847,
 'eval_runtime': 15.0913,
 'eval_samples_per_second': 13.253,
 'eval_steps_per_second': 1.657,
 'epoch': 1.0}
```

12) Step 12: Save and Load Model

در مراحل قبلی ذخیره داده‌ها انجام شده است.

۲. سوال ۲

در این سوال قصد داریم تا شما را با تسک word analogy آشنا کنیم. بدین منظور:

1. **ابتدا بایستی از یکی از مدل‌ها یا همان word vectorهای از قبل آموزش دیده شده در hugging face استفاده کنید. (مدل پیشنهادی: https://huggingface.co/fse/word2vec-google-news-300/tree/main)**
2. **با استفاده از معیار شباهت کسینوسی، شباهت‌ها (similarity) را محاسبه کنید.**
3. **از word embedding های بدست آمده برای حل مسائلی مثل:**
   Man is to Woman as King is to _____ (answer: queen)
   استفاده کنید.
4. ***(اختیاری)* word embeddingها را مدیفای کنید تا بایاس embeddingها را کاهش دهید.**

به نوت‌بوک سوال (Assignment 2.ipynb) مراجعه کنید و قسمت‌های خواسته شده را تکمیل کنید. برای هر بخش، نکات، درسنامه و راهنمایی‌های مرتبط قرار گرفته است.

پیاده‌سازی:

1) Step 1: Installing Libraries

ابتدا کتابخانه‌های مورد نیاز پروژه را نصب کردیم.

برای اجرای کدها، کتابخانه‌های genism و huggingface_hub نصب می‌شوند. Gensim برای کار با مدل‌های برداری زبان مانند Word2Vec کاربرد دارد و Hugging Face Hub امکان دانلود فایل‌ها و مدل‌ها را از مخزن آنلاین فراهم می‌کند. نصب این دو کتابخانه با دستور pip install! انجام می‌شود.

```
!pip install gensim huggingface_hub
import numpy as np
from gensim.models import KeyedVectors
from huggingface_hub import hf_hub_download
```

2) Step 2: Loading The Model

سپس بردارهای واژگان (word vectors) را بارگذاری می‌کنیم. در این تمرین، از بردارهای واژگانی با ۳۰۰ بعد استفاده خواهیم کرد که بر روی حدود ۱۰۰ میلیارد کلمه از مجموعه داده‌ی Google News آموزش دیده‌اند تا واژه‌ها را نمایش دهند. (می‌توان از مدل‌هایی با ابعاد کمتر مانند GloVe با ۵۰ بعد نیز استفاده کرد.)

برای اینکار ابتدا فایل های word2vec-google-news-300.model.vectors.npy و word2vec-google-news-300.model را از آدرس داده شده در huggingface دستی دانلود می‌کنیم.

مدلی که از طریق Gensim ذخیره شده، شامل دو فایل اصلی است: فایل model. که ساختار مدل، اطلاعات واژگان، تنظیمات و پارامترهای آموزش را نگه می‌دارد، و فایل vectors.npy. که حاوی بردارهای واقعی کلمات (embedding matrix) است. فایل اول برای بازسازی مدل و مدیریت عملیات مرتبط با واژه‌ها ضروری است، اما بدون فایل دوم، مدل به بردارهای عددی کلمات دسترسی نخواهد داشت؛ بنابراین هر دو فایل برای بارگذاری کامل و عملکرد صحیح مدل مورد نیازند.

```
# Path to the model file
model_path = "./Models/HuggingFace-GoogleNews/word2vec-google-news-300.model"

# Load the Gensim model
word_vectors = KeyedVectors.load(model_path)

# Create a dictionary mapping each word to its corresponding vector
word_to_vec_map = {word: word_vectors[word] for word in word_vectors.key_to_index}
```

3) Step 3: Checking examples

10 نمونه از بردارهای این مدل را نیز به کمک کد زیر چاپ کردیم:

```
# Show 10 word vectors and their lengths in a formatted table
print("\nSample word vectors:")

table_data = []
count = 0
for word in word_to_vec_map:
    if count >= 10:
        break
    vector = word_to_vec_map[word]
    short_vec = ", ".join(f"{v:.3f}" for v in vector[:5])  # first 5 numbers, rounded
    table_data.append([word, f"[{short_vec}, ...]", len(vector)])
    count += 1

# Define column headers
headers = ["Word", "Vector (first 5 values)", "Length"]

# Print as a table
print(tabulate(table_data, headers=headers, tablefmt="grid"))
```

```
Sample word vectors:
+--------+-----------------------------------------------+----------+
| Word   | Vector (first 5 values)                       |   Length |
+========+===============================================+==========+
| </s>   | [0.001, -0.001, 0.000, 0.002, 0.001, ...]     |      300 |
+--------+-----------------------------------------------+----------+
| in     | [0.070, 0.087, 0.088, 0.062, 0.069, ...]      |      300 |
+--------+-----------------------------------------------+----------+
| for    | [-0.012, -0.047, 0.045, 0.063, -0.018, ...]   |      300 |
+--------+-----------------------------------------------+----------+
| that   | [-0.016, -0.028, 0.083, 0.050, -0.110, ...]   |      300 |
+--------+-----------------------------------------------+----------+
| is     | [0.007, -0.073, 0.172, 0.023, -0.133, ...]    |      300 |
+--------+-----------------------------------------------+----------+
| on     | [0.027, -0.091, 0.028, 0.204, 0.006, ...]     |      300 |
+--------+-----------------------------------------------+----------+
| ##     | [0.169, -0.063, -0.000, 0.063, 0.087, ...]    |      300 |
+--------+-----------------------------------------------+----------+
| The    | [-0.173, 0.279, 0.107, -0.158, -0.084, ...]   |      300 |
+--------+-----------------------------------------------+----------+
| with   | [-0.025, 0.022, -0.035, 0.137, 0.016, ...]    |      300 |
+--------+-----------------------------------------------+----------+
| said   | [-0.009, -0.044, 0.100, -0.076, -0.057, ...]  |      300 |
+--------+-----------------------------------------------+----------+
```

4) Step 4 - Method 1: Cosine similarity

برای اندازه‌گیری میزان شباهت بین دو واژه، باید شباهت بین بردارهای تعبیه‌شده (embedding vectors) آن‌ها را محاسبه کنیم. اگر دو بردار خیلی مشابه باشند، زاویه‌ی بین‌شان کوچک است و کسینوس آن نزدیک به ۱ خواهد بود، اگر غیر مشابه باشند، زاویه بزرگ‌تر می‌شود و مقدار شباهت کسینوسی به صفر یا منفی نزدیک می‌شود. زاویه بین دو بردار می‌تواند معیاری برای سنجش میزان شباهت آن‌ها باشد.

در ادامه برای محاسبه مقادیر اختلاف در مثال های داده شده از کد زیر استفاده کردیم:

```
# GRADED FUNCTION: cosine_similarity

def cosine_similarity(u, v):

    dot = np.dot(u, v)                    # u . v

    norm_u = np.linalg.norm(u)            # ||u||
    norm_v = np.linalg.norm(v)            # ||v||

    cosine_similar = dot / (norm_u * norm_v)


    return cosine_similar
```

نمونه مثال ها و خروجی ها:

```
  father = word_to_vec_map["father"]
  mother = word_to_vec_map["mother"]
  ball = word_to_vec_map["ball"]
  crocodile = word_to_vec_map["crocodile"]
  big = word_to_vec_map["big"]
  bigger = word_to_vec_map["bigger"]
  small = word_to_vec_map["small"]
  smaller = word_to_vec_map["smaller"]

  print("cosine_similarity(father, mother) = ", cosine_similarity(father, mother))
  print("cosine_similarity(ball, crocodile) = ",cosine_similarity(ball, crocodile))
  print("cosine_similarity(big - bigger, small - smaller) = ",cosine_similarity(big - bigger, small - smaller))
✓ 0.0s
cosine_similarity(father, mother) =  0.79014826
cosine_similarity(ball, crocodile) =  0.10283585
cosine_similarity(big - bigger, small - smaller) =  0.5752201
```

در این قسمت برخلاف انتظار، خروجی cosine_similarity(big - bigger, small - smaller) منفی نشد.

برای همین اول برای اطمینان از اینکه مدل صحیح است همان مدل را از جای دیگر دانلود کردیم:

از طریق ریپازیتوری گیتهاب زیر:

https://github.com/mmihaltz/word2vec-GoogleNews-vectors/tree/master

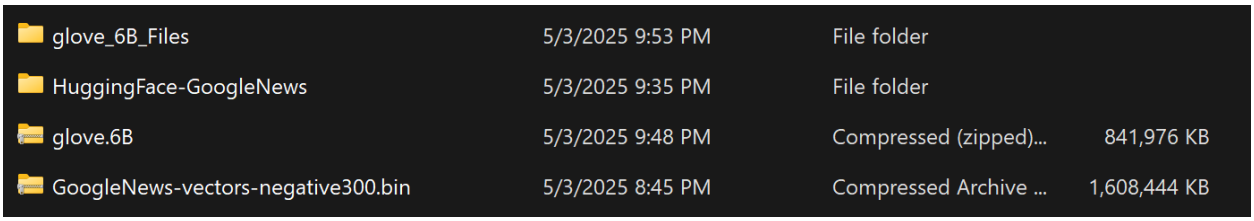
به فایل گوگل درایو این مدل دسترسی پیدا کردیم و آن را دانلود کردیم:

https://drive.google.com/file/d/0B7XkCwpI5KDYNlNUTTlSS21pQmM/edit?resourcekey=0-wjGZdNAUop6WykTtMip30g

و حتی با یک مدل دیگر هم تست کردیم با مدل Glove که یکی از پرکاربردترین مدل هاست:

`glove.6B.300d.txt`

https://nlp.stanford.edu/projects/glove/



مدل‌ها به دلیل حجم زیاد در کنار باقی فایلها در Quera آپلود نشدند.

همچنین بررسی های مختلفی را هم برای این نتیجه گیری انجام دادیم:

از تابع زیر استفاده کردیم، و زوایای بین آنها کمتر از ۹۰ درجه بودند و برای همین نتیجه برخلاف تصور بود و این نشاندهنده علت مثبت بودن اختلاف قبل است.

```python
def analyze_difference(w1, w2, w3, w4, word_vectors):
    diff1 = word_vectors[w1] - word_vectors[w2]
    diff2 = word_vectors[w3] - word_vectors[w4]

    cos_sim = cosine_similarity(diff1, diff2)
    angle_rad = np.arccos(np.clip(cos_sim, -1.0, 1.0))
    angle_deg = np.degrees(angle_rad)
    norm1 = np.linalg.norm(diff1)
    norm2 = np.linalg.norm(diff2)

    print(f"{w1}-{w2} vs {w3}-{w4}")
    print(f"  Cosine similarity: {cos_sim:.4f}")
    print(f"  Angle between: {angle_deg:.2f} degrees")
    print(f"  Norms: {norm1:.2f} vs {norm2:.2f}")
    print("-" * 40)
```

✓ 0.0s

```python
analyze_difference("big", "bigger", "small", "smaller", word_vectors)
```

✓ 0.0s

```
big-bigger vs small-smaller
  Cosine similarity: 0.5752
  Angle between: 54.88 degrees
  Norms: 1.96 vs 2.00
----------------------------------------
```

خلاصه نتیجه‌گیری:

Word2Vec و GloVe ذاتاً جهت‌های «big → bigger» و «small → smaller» را به‌عنوان جهت‌های مخالف رمزگذاری نمی‌کنند. در عوض، این مدل‌ها یک جابجایی برداری مشابه برای شکل مقایسه‌ای هر صفت یاد می‌گیرند، و به همین دلیل، شباهت کسینوسی در نمونه کد مقدار مثبتی دارد. این رفتار مورد انتظار است و از طریق ارزیابی‌های قیاسی نیز تأیید شده (یعنی مدل «smaller» را با «small» همان‌طور مرتبط می‌داند که «bigger» را با «big»)

برای به‌دست آوردن بردارهایی با جهت‌های واقعاً متضاد برای چنین جفت‌هایی، لازم است دانش معنایی بیرونی یا اهداف آموزشی تخصصی که این محدودیت‌ها را تحمیل می‌کنند، به مدل اضافه شود.

خلاصه اینکه، نتیجه مثبت شباهت کسینوسی بازتابی از این واقعیت است که این نوع توکـن‌گذاری‌های برداری بیشتر بر الگوهای زبانی منظم (مانند ساختار صرفی و زمینه توزیعی کلمات) تمرکز دارند تا بر محورهای قابل‌تعبیر برای متضادهای معنایی.

این نتایج بصورت واضح در منابع اصلی این ۲ روش نیز گذاشته شده‌اند:

1. Mikolov et al., 2013 – Efficient Estimation of Word Representations in Vector Space

arXiv:1301.3781

مقاله اصلی Word2Vec: در این مقاله، نویسندگان رابطه‌های برداری مثل "king - man + woman = queen" رو معرفی می‌کنن و نشون می‌دن که مدل می‌تونه الگوهای صرفی (morphological) مثل صفت ← تفضیلی رو یاد بگیره.

2. Pennington et al., 2014 – GloVe: Global Vectors for Word Representation

https://nlp.stanford.edu/pubs/glove.pdf

مقاله اصلی GloVe: اشاره می‌کنه که GloVe هم روابط معنایی رو به‌صورت بردارهای خطی encode می‌کنه، ولی تأکید داره که همه‌ی روابط در فضای برداری به یک محور معنایی تفسیرپذیر ختم نمی‌شن.

5) Step 5 – Method 2: Word analogy task

در این بخش میخواهیم مساله analogy را با استفاده از مدل برداری Word2Vec حل کنیم.

مسئله‌ی analogy به صورت (A به B همان‌گونه است که C به ؟) مطرح می‌شود، و ما می‌خواهیم با استفاده از بردارهای کلمات در فضای معنایی، کلمه‌ی چهارم را پیش‌بینی کنیم.

کار با استفاده از مدل‌های برداری کلمات (مثل Word2Vec) انجام می‌شود که در آن هر کلمه به‌صورت یک بردار عددی در فضای چندبعدی نمایش داده می‌شود و تفاوت بین دو بردار می‌تواند نشان‌دهنده یک رابطه معنایی (مثلاً مقایسه جنسیت، درجه، یا کشور-پایتخت) باشد.

در این تابع، ابتدا بردارهای کلمات A، B و C گرفته می‌شوند و سپس بردار اختلاف بین A و B به‌عنوان «جهت رابطه» در نظر گرفته می‌شود. بعد، برای همه کلمات موجود در مدل (به‌جز A، B و C)، تفاوت آن‌ها با C محاسبه می‌شود و شباهت کسینوسی این تفاوت با بردار رابطه اصلی بررسی می‌گردد.

در نهایت، کلمه‌ای که بیشترین شباهت کسینوسی را دارد به‌عنوان بهترین پاسخ انتخاب و بازگردانده می‌شود و می‌تواند روابط معنایی ظریف بین کلمات را در فضای برداری تشخیص دهد و به صورت خودکار کلمات مرتبط را پیش‌بینی کند.

```python
# GRADED FUNCTION: complete_analogy
def complete_analogy(word_a, word_b, word_c, word_to_vec_map):
    """
    Performs the word analogy task: a is to b as c is to ____.
    """

    # convert words to lower case
    word_a, word_b, word_c = word_a.lower(), word_b.lower(), word_c.lower()

    # Get the word embeddings e_a, e_b and e_c
    e_a, e_b, e_c = word_to_vec_map[word_a], word_to_vec_map[word_b], word_to_vec_map[word_c]

    words = word_to_vec_map.keys()
    max_cosine_sim = -100
    best_word = None

    # loop over the whole word vector set
    for w in words:
        if w in [word_a, word_b, word_c]:
            continue

        # Compute cosine similarity
        cosine_sim = cosine_similarity(e_b - e_a, word_to_vec_map[w] - e_c)

        if cosine_sim > max_cosine_sim:
            max_cosine_sim = cosine_sim
            best_word = w

    return best_word
```

نمونه های خروجی های گرفته شده:

```python
triads_to_try = [('france', 'paris', 'germany'), ('man', 'woman', 'boy'), ('small', 'smaller', 'large'),
                 ('walk', 'walking', 'swim'), ('he', 'him', 'she'),('bad', 'worse', 'good')]
for triad in triads_to_try:
    print ('{} -> {} :: {} -> {}'.format( *triad, complete_analogy(*triad,word_to_vec_map)))
```

✓ 4m 29.8s

```
france -> paris :: germany -> plaster
man -> woman :: boy -> girl
small -> smaller :: large -> larger
walk -> walking :: swim -> swimming
he -> him :: she -> her
bad -> worse :: good -> better
```

6) Step 6 – Method 3: Debiasing word vectors (OPTIONAL/UNGRADED)

در این بخش از تمرین، قرار است مسئله‌ی سوگیری جنسیتی در توکن‌گذاری کلمات را بررسی کنیم و با الگوریتم‌هایی برای کاهش این سوگیری آشنا شویم. علاوه بر آشنایی با مفهوم debiasing، این تمرین به ما کمک می‌کند تا درک دقیق‌تری از عملکرد بردارهای کلمات به دست آوریم.

در ابتدا بررسی می‌کنیم که بردارهای GloVe چگونه با مفهوم جنسیت مرتبط هستند و چگونه می‌توانیم با استفاده از آن‌ها، به‌طور تقریبی مفهوم "جنسیت" را کدگذاری کنیم. این بردار، تفاوت جنسیتی میان واژه‌ها را نشان می‌دهد.

گرچه می‌توانیم با ترکیب چند جفت واژه مانند مادر - پدر، دختر - پسر و محاسبه‌ی میانگین آن‌ها، به برداری دقیق‌تر دست پیدا کنیم، اما استفاده از جفت زن - مرد نیز نتایج قابل قبولی ارائه می‌دهد و برای شروع مناسب است. این بخش می‌تواند شامل مقدار کمی جبر خطی هم باشد.

- **مرحله اول: استخراج جهت جنسیت (Gender Direction)**

خروجی‌ای که برای g = word_to_vec_map['woman'] - word_to_vec_map['man'] دریافت می‌کنیم یک بردار با ۳۰۰ مقدار عددی (floats) هست.

```
g = word_to_vec_map['woman'] - word_to_vec_map['man']
print(g)
```

✓ 0.0s

```
[-8.30078125e-02 -2.08007812e-01 -1.37695312e-01 -2.44140625e-02
 2.83203125e-02 -6.61621094e-02  8.39843750e-02  1.87377930e-02
-3.17382812e-02  4.66384888e-02 -1.25976562e-01 -9.37500000e-02
 2.09960938e-02  2.63671875e-02  7.49511719e-02  5.34667969e-02
-8.98437500e-02 -1.12792969e-01  3.27148438e-02  7.00683594e-02
 1.01806641e-01 -4.93927002e-02 -8.12988281e-02 -1.36718750e-01
-5.38330078e-02 -4.10156250e-02 -9.27734375e-02  3.90625000e-03
-2.53906250e-02 -1.66503906e-01 -1.85546875e-02 -1.42944336e-01
-1.61621094e-01 -8.48388672e-02 -1.95312500e-02 -1.30859375e-01
 5.85937500e-02  7.81250000e-02 -1.67480469e-01 -1.21093750e-01
-1.88720703e-01  1.04980469e-02 -5.37109375e-02 -1.61743164e-02
```

در اینجا داریم بررسی می‌کنیم که هر اسم داخل لیست، چقدر با بردار جنسیت g هم‌جهت هست.

معنی مقدار شباهت کسینوسی (cosine similarity) که چاپ می‌شه:

- **مقدار نزدیک به +۱** :این اسم در فضای برداری بیشتر به سمت "زن" جهت‌گیری داره.
- **مقدار نزدیک به −۱** :این اسم بیشتر در جهت "مرد" قرار داره.
- **مقدار نزدیک به ۰** :این اسم جهت جنسیتی مشخصی نداره یا خنثی است.

```
print ('List of names and their similarities with constructed vector:')

# girls and boys name
name_list = ['john', 'marie', 'sophie', 'ronaldo', 'priya', 'rahul', 'danielle', 'reza', 'katy', 'yasmin']

for w in name_list:
    print (w, cosine_similarity(word_to_vec_map[w], g))
```

✓ 0.0s

```
List of names and their similarities with constructed vector:
john -0.03143504
marie 0.17699815
sophie 0.1283398
ronaldo -0.09830637
priya 0.09364402
rahul -0.036300182
danielle 0.10128634
reza -0.056654923
katy 0.09632665
yasmin 0.19511871
```

در ادامه در این بخش از کد، داریم بررسی می‌کنیم که مدل Word2Vec چه میزان سوگیری جنسیتی نسبت به برخی از کلمات عمومی، شغلی و فرهنگی دارد. با استفاده از بردار g که قبلاً به عنوان جهت جنسیت (difference between "woman" and "man") تعریف کردیم، حالا برای هر کلمه‌ی موجود در word_list مقدار شباهت کسینوسی (cosine similarity) آن با g را محاسبه می‌کنیم.

این کد به ما نشون می‌ده که هر کلمه‌ی موجود در لیست، چقدر در فضای برداری به سمت "زنانه" یا "مردانه" بودن تمایل دارد.

- اگر مقدار شباهت کسینوسی **مثبت و نزدیک به ۱** باشد، یعنی آن کلمه در مدل بیشتر **زنانه** تلقی شده.
- اگر مقدار **منفی و نزدیک به -۱** باشد، آن کلمه در مدل بیشتر **مردانه** تلقی شده.
- اگر مقدار نزدیک به صفر باشد، یعنی مدل آن را خنثی از نظر جنسیت در نظر گرفته.

```
print('Other words and their similarities:')
word_list = ['lipstick', 'guns', 'science', 'arts', 'literature', 'warrior','doctor', 'tree', 'receptionist',
             'technology',  'fashion', 'teacher', 'engineer', 'pilot', 'computer', 'singer']
for w in word_list:
    print (w, cosine_similarity(word_to_vec_map[w], g))
```

✓ 0.0s

```
Other words and their similarities:
lipstick 0.26272777
guns -0.12910882
science 0.038018893
arts 0.13724276
literature 0.18530814
warrior -0.09518411
doctor 0.16264015
tree 0.063181736
receptionist 0.3256916
technology 0.06544881
fashion 0.16653466
teacher 0.14762524
engineer -0.057295308
pilot 0.036814738
computer 0.09310092
singer 0.15405332
```

با بررسی این شباهت‌ها، می‌تونیم ببینیم آیا مدل دارای کلیشه‌های جنسیتی هست یا نه. برای مثال:

- آیا کلماتی مثل "receptionist" یا "lipstick" در جهت زنانه قرار گرفتن؟
- آیا "engineer" یا "pilot" بیشتر در جهت مردانه هستن؟

اگر چنین الگوهایی دیده بشن، یعنی مدل زبانی (و داده‌هایی که باهاش آموزش دیده) دارای سوگیری جنسیتی هست — که می‌تونه در کاربردهای واقعی منجر به تصمیم‌گیری‌های ناعادلانه بشود.

تا این مرحله از کار، توانسته‌ایم مراحل کلیدی را به‌درستی اجرا کنیم. ابتدا بردار جنسیت را به صورت:

$$g = e_{woman} - e_{man}$$

استخراج کردیم. سپس شباهت کسینوسی میان اسامی و واژه‌های مختلف با این بردار را محاسبه نمودیم و مشاهده کردیم که برخی از نتایج با کلیشه‌های جنسیتی رایج هم‌راستا هستند؛ برای مثال، واژه‌ی «computer» به «man» و واژه‌ی «literature» به «woman» نزدیک‌تر بود. اکنون قصد داریم وارد مرحله‌ی **خنثی‌سازی (neutralization)** شویم.

در این مرحله، می‌خواهیم تابعی به نام `neutralize()` پیاده‌سازی کنیم که سوگیری جنسیتی را از کلمات خنثی مانند"receptionist"، "doctor"، "scientist"، "teacher"و واژه‌هایی از این دست حذف کند. هدف ما این است که بردار چنین واژه‌هایی دیگر در راستای بردار جنسیت قرار نداشته باشد و تنها معنای خنثی آن‌ها در فضای برداری باقی بماند.

```python
def neutralize(word, g, word_to_vec_map):
    """
    Removes the bias of "word" by projecting it on the space orthogonal to the bias axis.
    This function ensures that gender neutral words are zero in the gender subspace.

    Arguments:
        word -- string indicating the word to debias
        g -- numpy-array of shape (50,), corresponding to the bias axis (such as gender)
        word_to_vec_map -- dictionary mapping words to their corresponding vectors.

    Returns:
        e_debiased -- neutralized word vector representation of the input "word"
    """
    e = word_to_vec_map[word]
    e_biascomponent = np.dot(e, g) / np.linalg.norm(g)**2 * g
    e_debiased = e - e_biascomponent

    return e_debiased
```

فرآیند خنثی‌سازی شامل سه گام اصلی است:

۱. **دریافت بردار کلمه:**
ابتدا بردار مربوط به واژه‌ی مورد نظر (برای مثال، "receptionist") را از مدل واژه‌برداری دریافت می‌کنیم.

۲. **محاسبه مؤلفه‌ی جنسیتی:**
سپس بخش مربوط به جنسیت را که در راستای بردار جنسیت gقرار دارد، محاسبه می‌کنیم. این مؤلفه همان پروژکشن بردار واژه روی محور جنسیت است که نشان می‌دهد چه بخشی از معنای واژه با مفهوم جنسیتی هم‌راستا است.

۳. **حذف مؤلفه‌ی جنسیتی:**

در نهایت، مؤلفه‌ی جنسیتی را از بردار اصلی کسر می‌کنیم تا تنها مؤلفه‌ی عمود بر بردار جنسیت باقی بماند؛ به‌عبارت دیگر، بردار جدید تنها در زیرفضایی قرار می‌گیرد که نسبت به جنسیت بی‌طرف است.

مثال اجرا شده:

```
e = "receptionist"
print("cosine similarity between " + e + " and g, before neutralizing: ", cosine_similarity(word_to_vec_map["receptionist"], g))

e_debiased = neutralize("receptionist", g, word_to_vec_map)
print("cosine similarity between " + e + " and g, after neutralizing: ", cosine_similarity(e_debiased, g))
```

✓ 0.0s

```
cosine similarity between receptionist and g, before neutralizing:  0.3256916
cosine similarity between receptionist and g, after neutralizing:  1.1497245e-08
```

- **مرحله دوم: الگوریتم هم‌ترازی (Equalization) برای کلمات خاص جنسیتی**

در ادامه در این مرحله می‌خواهیم با استفاده از الگوریتم هم‌ترازی(Equalization) ، جفت‌واژه‌های جنسیتی مانند «actor» و «actress» را به‌گونه‌ای تنظیم کنیم که تنها در ویژگی جنسیت با یکدیگر تفاوت داشته باشند و در سایر ابعاد معنایی برابر در نظر گرفته شوند. هدف ما این است که این کلمات نسبت به واژه‌های خنثی‌شده‌ای مانند «receptionist» یا «scientist» فاصله‌ای یکسان داشته باشند و هیچ‌کدام به آن‌ها نزدیک‌تر نباشد. حتی در شرایطی که سوگیری جنسیتی یک واژه مانند «babysit» کاهش یافته باشد، ممکن است همچنان به یکی از این دو کلمه نزدیک‌تر باشد و این موضوع می‌تواند نوعی کلیشه پنهان را در فضای برداری حفظ کند. بنابراین، با به‌کارگیری الگوریتم Equalization تلاش می‌کنیم مؤلفه‌های جنسیتی این دسته از واژه‌ها را بازتنظیم کنیم تا نمایش معنایی آن‌ها در مدل به شکلی متعادل و بی‌طرفانه بازتاب یابد.

بنابراین می‌خواهیم کدی پیاده‌سازی کنیم که جفت‌کلمات جنسیتی مانند «actor» و «actress» را به‌گونه‌ای تنظیم کند که **تنها تفاوت آن‌ها در جنسیت باشد** و در سایر ویژگی‌های معنایی با یکدیگر **برابر و متقارن** باشند. این کار برای جلوگیری از تقویت کلیشه‌های جنسیتی در مدل‌های زبانی انجام می‌شود.

```
def equalize(pair, bias_axis, word_to_vec_map):

    ### START CODE HERE ###
    w1, w2 = pair
    e_w1, e_w2 = word_to_vec_map[w1], word_to_vec_map[w2]

    # Step 2: Compute the mean vector
    mu = (e_w1 + e_w2) / 2

    # Step 3: Compute projections
    mu_B = np.dot(mu, bias_axis) / np.linalg.norm(bias_axis)**2 * bias_axis
    mu_perp = mu - mu_B

    # Step 4: Compute bias components of e_w1 and e_w2
    e_w1B = np.dot(e_w1, bias_axis) / np.linalg.norm(bias_axis)**2 * bias_axis
    e_w2B = np.dot(e_w2, bias_axis) / np.linalg.norm(bias_axis)**2 * bias_axis

    # Step 5: Correct the bias components
    corrected_e_w1B = np.sqrt(abs(1 - np.linalg.norm(mu_perp)**2)) * (e_w1B - mu_B) / np.linalg.norm((e_w1 - mu_perp) - mu_B)
    corrected_e_w2B = np.sqrt(abs(1 - np.linalg.norm(mu_perp)**2)) * (e_w2B - mu_B) / np.linalg.norm((e_w2 - mu_perp) - mu_B)

    # Step 6: Combine corrected bias components with orthogonal part
    e1 = corrected_e_w1B + mu_perp
    e2 = corrected_e_w2B + mu_perp
    ### END CODE HERE ###

    return e1, e2
```

مراحل کد بالا به شرح زیر است:

۱. **دریافت بردارهای دو کلمه**
ابتدا بردارهای معنایی دو واژه‌ی جنسیتی مانند «actor» و «actress» از مدل تعبیه واژه دریافت می‌شود.

۲. **محاسبه بردار میانگین (میان دو واژه)**
بردار میانگین این دو کلمه محاسبه می‌شود تا موقعیت مرکزی آن‌ها در فضای معنایی مشخص شود.

۳. **تفکیک میانگین به دو بخش: جنسیتی و خنثی**
بردار میانگین به دو مؤلفه تقسیم می‌شود:
   - مؤلفه‌ای که در جهت بردار جنسیت قرار دارد (یعنی بخش سوگیری)
   - مؤلفه‌ای که عمود بر بردار جنسیت است (یعنی بخش خنثی)

۴. **محاسبه مؤلفه جنسیتی هر واژه به‌صورت جداگانه**
مشخص می‌کنیم هر واژه دقیقاً تا چه اندازه در راستای جنسیت زن یا مرد قرار دارد.

۵. **تنظیم مؤلفه‌های جنسیتی برای ایجاد تقارن**
مؤلفه‌های جنسیتی هر دو واژه به‌گونه‌ای بازتنظیم می‌شوند که فاصله‌شان از مرکز (در بعد جنسیت) برابر ولی در جهت مخالف باشد. این کار باعث تقارن در راستای جنسیت می‌شود.

۶. **ترکیب مؤلفه‌های اصلاح‌شده با بخش خنثی**
در نهایت، هر مؤلفه جنسیتی اصلاح‌شده با بخش خنثی ترکیب می‌شود تا بردار نهایی برای هر واژه ساخته شود.

یک نمونه مثال اجرا شده در قسمت پایین آورده‌ایم:

```
print("cosine similarities before equalizing:")
print("cosine_similarity(word_to_vec_map[\"man\"], gender) = ", cosine_similarity(word_to_vec_map["man"], g))
print("cosine_similarity(word_to_vec_map[\"woman\"], gender) = ", cosine_similarity(word_to_vec_map["woman"], g))
print()
e1, e2 = equalize(("man", "woman"), g, word_to_vec_map)
print("cosine similarities after equalizing:")
print("cosine_similarity(e1, gender) = ", cosine_similarity(e1, g))
print("cosine_similarity(e2, gender) = ", cosine_similarity(e2, g))
```

✓ 0.0s

```
cosine similarities before equalizing:
cosine_similarity(word_to_vec_map["man"], gender) =  -0.15934694
cosine_similarity(word_to_vec_map["woman"], gender) =  0.5120308

cosine similarities after equalizing:
cosine_similarity(e1, gender) =  -0.66846603
cosine_similarity(e2, gender) =  0.668466
```

در این آزمایش، ابتدا شباهت کسینوسی کلمات "man" و "woman" نسبت به بردار جنسیت محاسبه شد و نتایج نشان داد که "woman" به‌طور قابل توجهی در راستای جنسیت زنانه قرار دارد، در حالی‌که "man" فاصله‌ای کمتر و در جهت مخالف دارد. این عدم تعادل نشان‌دهنده‌ی نوعی سوگیری در نمایش برداری این کلمات بود. با اجرای تابع equalize()، این دو بردار به‌گونه‌ای بازتنظیم شدند که پس از هم‌ترازی، هر دو به یک اندازه و در جهت‌های متقارن نسبت به بردار جنسیت قرار گرفتند. این نتیجه نشان می‌دهد که الگوریتم به‌درستی توانسته تفاوت جنسیتی این دو واژه را حفظ کند، در حالی‌که تعادل و بی‌طرفی نسبت به سایر مفاهیم را برقرار کرده است.

نمونه مثال‌های دیگر و اعمال خلاصه روندهای این بخش:

در بخش اول، سه واژه‌ی نسبتاً خنثی شامل"scientist" ، "nurse"و "engineer" را انتخاب کردیم تا روی آن‌ها الگوریتم خنثی‌سازی (neutralize) را اعمال کنیم. ابتدا شباهت کسینوسی هر واژه با بردار جنسیت) که از تفاوت بین "woman" و "man" به‌دست آمده بود (محاسبه شد. سپس با اجرای تابع neutralize()، مؤلفه‌ای از این بردار که در راستای جنسیت قرار داشت حذف شد و دوباره شباهت کسینوسی بررسی گردید. انتظار ما این بود که پس از خنثی‌سازی، این شباهت به صفر نزدیک شود؛ به این معنا که واژه دیگر با جنسیت خاصی هم‌راستا نباشد و سوگیری ناخواسته در فضای معنایی آن کاهش یابد.

در بخش دوم، دو جفت‌واژه‌ی مشخصاً جنسیتی شامل ("king", "queen") و ("brother", "sister") مورد بررسی قرار گرفتند تا روی آن‌ها الگوریتم هم‌ترازی (equalize) اعمال شود. در ابتدا شباهت کسینوسی هر واژه با بردار جنسیت محاسبه شد و سپس با استفاده از تابع equalize()، این واژه‌ها به‌گونه‌ای بازتنظیم شدند که تنها تفاوت آن‌ها در بعد جنسیت باقی بماند و نسبت به واژه‌های خنثی‌شده، موقعیتی متقارن داشته باشند. انتظار ما این بود که بعد از هم‌ترازی، دو کلمه به یک اندازه اما در جهت‌های مخالف نسبت به بردار جنسیت قرار بگیرند (مثلاً یکی با شباهت +۰٫۶۶ و دیگری با -۰٫۶۶) که این موضوع نشان‌دهنده موفقیت در حذف سوگیری نابرابر و حفظ تقارن معنایی است.

```
print("==== Testing neutralize() on neutral words ====\n")
neutral_words = ["scientist", "nurse", "engineer"]

for word in neutral_words:
    before = cosine_similarity(word_to_vec_map[word], g)
    e_debiased = neutralize(word, g, word_to_vec_map)
    after = cosine_similarity(e_debiased, g)
    print(f"Word: {word}")
    print(f"  Cosine similarity before neutralizing: {before:.6f}")
    print(f"  Cosine similarity after neutralizing:  {after:.6f}")
    print("---")

print("\n==== Testing equalize() on gendered word pairs ====\n")
equalize_pairs = [("king", "queen"), ("brother", "sister")]

for w1, w2 in equalize_pairs:
    before_1 = cosine_similarity(word_to_vec_map[w1], g)
    before_2 = cosine_similarity(word_to_vec_map[w2], g)
    e1, e2 = equalize((w1, w2), g, word_to_vec_map)
    after_1 = cosine_similarity(e1, g)
    after_2 = cosine_similarity(e2, g)
    print(f"Pair: ({w1}, {w2})")
    print(f"  Cosine similarity before equalizing: {w1} = {before_1:.6f}, {w2} = {before_2:.6f}")
    print(f"  Cosine similarity after equalizing:  {w1} = {after_1:.6f}, {w2} = {after_2:.6f}")
    print("---")
```

نتایج:

```
==== Testing neutralize() on neutral words ====

Word: scientist
  Cosine similarity before neutralizing: 0.026401
  Cosine similarity after neutralizing:  -0.000000
---
Word: nurse
  Cosine similarity before neutralizing: 0.337688
  Cosine similarity after neutralizing:  -0.000000
---
Word: engineer
  Cosine similarity before neutralizing: -0.057295
  Cosine similarity after neutralizing:  0.000000
---

==== Testing equalize() on gendered word pairs ====

Pair: (king, queen)
  Cosine similarity before equalizing: king = -0.109334, queen = 0.263174
  Cosine similarity after equalizing:  king = -0.384879, queen = 0.384879
---
Pair: (brother, sister)
  Cosine similarity before equalizing: brother = -0.113682, sister = 0.316114
  Cosine similarity after equalizing:  brother = -0.459553, sister = 0.459553
---
```

تحلیل نتایج:

بخش ۱ `neutralize()` روی کلمات خنثی

1. scientist

- قبل از :neutralizing ۰٫۰۲۶۴۰۱ → خیلی نزدیک به صفر
- بعد از :neutralizing تقریباً صفر کامل(`-0.000000`)

نتیجه :بردار کاملاً خنثی شد؛ همانطور که انتظار می‌رفت.

2. nurse

- قبل از :neutralizing ۰٫۳۳۷۶۸۸ → نشان‌دهنده‌ی تمایل به جنسیت زنانه
- بعد از :neutralizing صفر کامل

نتیجه :سوگیری حذف شده؛ الگوریتم مؤثر عمل کرده.

3. engineer

- قبل از → neutralizing: -0.057295 کمی تمایل به جنسیت مردانه
- بعد از :neutralizing صفر کامل

نتیجه :باز هم موفقیت‌آمیز؛ سوگیری جنسیتی برطرف شده.

جمع‌بندی این بخش:

الگوریتم neutralize()عملکرد خوبی داشته و سوگیری جنسیتی را از واژه‌های نسبتاً خنثی حذف کرده. بعد از اجرای آن، شباهت کسینوسی به‌درستی به صفر رسیده است.

بخش ۲ equalize() روی جفت‌کلمات جنسیتی

1. (king, queen)

- قبل از:equalizing
  - king = -0.109
  - queen = +0.263

→فاصله نامتقارن از محور جنسیت

- بعد از:equalizing
  - king = -0.384
  - queen = +0.384

نتیجه :تقارن کامل برقرار شده؛ موفقیت‌آمیز.

2. (brother, sister)

- قبل از:equalizing
  - brother = -0.113
  - sister = +0.316

→باز هم فاصله نامتقارن

- بعد از:equalizing
  - brother = -0.459
  - sister = +0.459

نتیجه :تقارن در راستای جنسیت با موفقیت برقرار شده.

جمع‌بندی این بخش:

الگوریتم equalize()نیز بسیار خوب عمل کرده. تفاوت معنایی بین دو واژه فقط به جنسیت محدود شده و فاصله‌شان از محور جنسیت دقیقاً برابر و در جهت مخالف شده است.